

Алгоритмы и структуры данных, 1 курс, 2 модуль ПМИ ФКН ВШЭ

Михаил Густокашин, 2025

`mgustokashin@hse.ru`

Оглавление

1	Алгоритмы и их сложность	5
1.1	Алгоритмы и структуры данных	5
1.2	Эффективность и RAM-машина	5
1.3	Оценка сложности программы	6
1.4	О-большое	7
1.5	Амортизированная сложность	9
1.6	Пространственная сложность	11
1.7	Тестирующая система	11
1.8	Самостоятельное тестирование	12
2	Стеки, очереди и деки	13
2.1	Стек	13
2.2	Правильная скобочная последовательность	14
2.3	Инфиксная и постфиксная запись	15
2.4	Монотонные стеки	18
2.5	Стек вызовов	20
2.6	Очередь	22
2.7	Дек	23
2.8	Минимум в окне	23
3	Квадратичные сортировки и сортировка слиянием	27
3.1	Сортировка пузырьком	27
3.2	Сортировка вставками	28
3.3	Сортировка выбором	29
3.4	Сравнение квадратичных сортировок	29
3.5	Сложность сортировок	30
3.6	Слияние отсортированных последовательностей	31
3.7	Сортировка слиянием	32
3.8	Подсчет числа инверсий	33
4	Куча, пирамидальная сортировка и бинарный поиск	35
4.1	Дерево	35
4.2	Куча	36
4.3	Представление кучи в виде массива	38
4.4	Пирамидальная сортировка	38
4.5	Бинарный поиск	41
4.6	Бинарный поиск по ответу	43

4.7	Бинарный поиск в массиве	45
4.8	Вещественный бинарный поиск	46

Лекция 1

Алгоритмы и их сложность

1.1 Алгоритмы и структуры данных

Алгоритм — это пошаговая последовательность инструкций для исполнителя, которые приводят к решению задачи. В нашем случае исполнителем является компьютер, а инструкция должна быть написана на понятном ему языке, в нашем случае это Python или C++.

Структура данных — это способ организации хранения и доступа к данным. Структуры данных позволяют организовать эффективный доступ к данным.

Хороший программист должен уметь делать следующее:

- Писать работающий код
- Уметь тестировать свой код
- Писать эффективный код
- Делать всё это быстро

На нашем курсе алгоритмов мы научимся выполнять все эти пункты. Алгоритмы и структуры данных позволят добиться эффективности, а все остальные пункты будут развиваться в процессе практической реализации изучаемых алгоритмов и структур данных.

1.2 Эффективность и RAM-машина

Писать работающий, протестированный код быстро — это уже большое достижение для программиста. Писать эффективный код нужно только хорошим программистам, работающим с большими объёмами данных или реализующим функции, которые будут вызываться огромное количество раз. Вам предстоит стать хорошими программистами.

Лучшей оценкой скорости работы программы является её непосредственный запуск на реальных данных и измерение времени работы. Однако, реализация программы может быть долгой, а скорость её работы неудовлетворительной. Кроме того, злоумышленники или конкуренты могут подобрать для нашей программы такие данные, на

которых она будет работать очень долго или потреблять очень много памяти, что приведёт к расходам и репутационным потерям. Ещё интересен вопрос: во сколько раз замедлится наша программа если объём данных возрастет в 10 раз? В 100 раз?

Для ответа на все эти вопросы было бы полезно иметь инструмент оценки времени работы программы до её написания. Современный компьютер устроен очень сложно: в нём есть кеши, предсказатели ветвлений, другие одновременно запущенные программы и другие факторы, значительно влияющие на время выполнения нашей программы.

Поэтому мы введём упрощённую модель, которая, тем не менее, очень сильно напоминает компьютер: RAM-машина. RAM — это Random Access Memory, произвольный доступ к памяти. То есть наша машина может обратиться к любой ячейке памяти за одну операцию. Это соответствует оперативной памяти компьютера и не соответствует, например SSD или HDD, где последовательный доступ к памяти гораздо быстрее произвольного.

Для простоты будем считать, что за одну операцию RAM-машина может выделять, читать и писать в одну ячейку памяти. В современных компьютерах под «ячейкой памяти» мы можем понимать машинное слово, равное 32 или 64 битам. Поэтому, например, записать в память очень длинное число, это не одна операция, а несколько. Количество операций пропорционально количеству цифр в таком числе. Кроме того, RAM-машина за одну операцию может считывать и писать число или символ из консоли в память, производить простейшие арифметические, битовые и логические операции над числами, сохранёнными в ячейках памяти.

В реальном компьютере не все операции одинаковы, например, умножение сложнее сложения, а деление сложнее умножения. Эти операции занимают разное количество тактов. Но для простоты будем считать, что у нас эти операции занимают одинаковое время.

Теперь у нас есть модель, в которой мы можем измерять время не в секундах, а в элементарных операциях. Мы написали некоторую программу и для ввода длины n можем посчитать количество операций, которая совершит наша программа, обозначим его за $T(n)$. И чем оно больше, тем, в общем случае, программа будет работать дольше на нашем компьютере.

1.3 Оценка сложности программы

Вернемся к вопросу о том, насколько замедлится наша программа, если увеличить ввод в 10 раз. Если мы нашли функцию для оценки времени работы программы для числа n и эта функция $T(n)$, то ответ очевиден, при увеличении ввода в 10 раз время работы программы станет равно $T(10n)$.

Рассмотрим пример программы для поиска максимума в последовательности чисел. Будем считать количество операций небрежно, поскольку, как мы увидим далее, это не будет сильно влиять на результат.

```
n = int(input()) # 1
a = list(map(int, input().split())) # n
now_max = a[0] # 1
for i in range(n): # n
    if a[i] > now_max: # 1
        now_max = a[i] # 1
```

```
print(now_max) # 1
```

В программе к каждой строке указана, сколько элементарных операций уйдёт на выполнение этой строки. Аккуратно просуммировав наши операции мы получим $T(n) = 1 + n + n + n + n + 1 = 2 + 4n$. n повторяется 4 раза, т.к. у нас есть чтение n чисел, цикл из n шагов (прибавление единицы к счётчику), n раз будет происходить выполнение строки с `if` и n раз будет происходить присваивание. Присваивание может происходить не n раз, а меньше, ведь очередное число не обязательно будет больше текущего максимума. Более того, оно вообще может не произойти ни разу. Таким образом $2 + 3n \leq T(n) \leq 2 + 4n$. Внимательный читатель может найти ещё много изъянов в этом подсчёте, например, в строке с `if` происходит обращение к двум ячейкам памяти и сравнение — это уже три операции. Более того, обращение к элементу массива, вообще говоря, тоже требует больше одной операции: необходимо взять ячейку памяти с «указателем» на начало массива, взять индекс, умножить индекс на размер ячейки и прибавить его к указателю на начало массива. Но не будем углубляться так сильно и примем $T(n) = 2 + 4n$. Это «худший» случай, когда `if` выполняется каждый раз. Например, это возможно, если вводится возрастающая последовательность. В реальной жизни худшие случаи будут генерировать злоумышленники, а в учебных задачах — их составители. Поэтому мы почти всегда будем ориентироваться на худший случай.

Итак, $T(n) = 2 + 4n$. При достаточно больших n можно сказать, что $T(n) = 4n$. Легко вычислить что $T(10n) = 40n$, что даёт нам соотношение $T(10n) = 10 \times T(n)$. Это линейное соотношение, поэтому такой алгоритм называется линейным. Как видно, константа 4 не играет здесь никакой роли, вместо неё можно взять абсолютно любую другую константу. Независимо от её значения мы определили главное: при увеличении входных данных в x раз программа замедлится в x раз. Такие алгоритмы называют алгоритмами с линейной сложностью.

Рассмотрим другой классический алгоритм, сортировку пузырьком.

```
for i in range(n):
    for j in range(n - 1):
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
```

В этот раз не будем точно считать количество действий, просто обозначим его за некоторую константу c . Количество действий, совершаемых алгоритмом для сортировки массива длиной n примерно равно $T(n) = c \times n^2$. Что будет, если увеличить размер ввода в 10 раз? $T(10n) = c \times (10n)^2 = c \times 100n^2$. Алгоритм замедлился в 100 раз при увеличении размера входа в 10 раз. Такие алгоритмы называются квадратичными.

Чтобы придать стройности теории сложности воспользуемся удобными обозначениями из математического анализа.

1.4 О-большое

Как мы увидели на примерах, константа не играет большой роли для оценки замедления программы при росте размера входных данных. В математическом анализе существует удобный инструмент для обозначения семейства функций, которые асимптотически (т.е. при очень больших n) имеют схожую степень роста. Этот инструмент — O (О-большое).

Напомним формальное определение: $f(n) = O(g(n))$ если существует такое N_0 , что при $n > N_0$ существует такая константа $c > 0$, что $f(n) \leq c \times g(n)$.

С помощью этого определения мы можем воспользоваться простейшими функциями для определения того класса сложности, к которой относится алгоритм. Например, если мы тщательно подсчитали сложность нашего алгоритма и она оказалась равна $T(n) = 4n^2 + 3n + 8.5$ то мы можем сказать, что $T(n) = O(n^2)$. Действительно, мы можем взять константу $c = 10$ и $N_0 = 10$ и определение будет выполнено. Таким образом, мы можем оценивать степень замедления нашего алгоритма независимо от деталей его реализации и конкретных констант.

При этом у нас пропали все младшие степени n . Вообще говоря, для $T(n) = 4n^2 + 3n + 8.5$ верны и следующие утверждения: $T(n) = O(n^2 + n)$ и $T(n) = O(4n)$. Хотя эти утверждения и верны, писать так не принято. Константы не пишут, т.к. в определении и так есть константа c , а младшие степени полинома не пишут, потому что где-то там при бесконечно большом n они уже не оказывают практически никакого влияния.

Самое забавное, что при $T(n) = n^2$ формально верно что $T(n) = O(n^3)$. Действительно, при $c = 1$ и $N_0 = 1$ определение верно. Вот только практической пользы для оценке сложности алгоритмов в этом нет, поэтому мы будем стараться найти самую маленькую функцию, которая ограничивает сложность алгоритма.

Рассмотрим еще один пример:

```
d = [] # 1
n = int(input()) # 1
for i in range(n): # n^2
    d.append(list(map(int, input().split())))
for via in range(n): # n^3
    for fr in range(n):
        for to in range(n):
            d[fr][to] = min(d[fr][to], d[fr][via] + d[via][to])
```

Эта программа считывает матрицу смежности графа и считает матрицу кратчайших расстояний с помощью алгоритма Флойда, но это неважно. Важно, что в ней есть блоки, работающие с разной сложностью. Создание пустого списка и считывание одного числа выполняется за $O(1)$ — так обозначают сложность алгоритмов, которые не зависят от входного параметра. Действительно, дадут ли нам матрицу 10×10 или 1000×1000 , сложность создания пустого списка не изменится — это будет занимать одну элементарную операцию. Сложность таких алгоритмов принято обозначать за $O(1)$.

Чтение матрицы занимает время $O(n^2)$. Несмотря на то, что там всего один цикл, идущий до n , внутри происходит считывание n значений из строки. А вычисление матрицы кратчайших путей $O(n^3)$. То есть сложность нашего алгоритма в целом будет $T(n) = c_1 \times n^3 + c_2 \times n^2 + c_3$. Но при действительно больших n какими бы большими не были константы у чтения данных или создания пустого списка, их влияние на итоговое время работы программы будет незначительным. Таким образом, мы получаем наглядное объяснение математическому факту $O(f(n) + g(n)) = O(\max(f(n), g(n)))$. На асимптотическую сложность алгоритма в целом влияет только его асимптотически самая сложная часть.

Вообще говоря, при оценки сложности алгоритмов используют и o (о-малое), тоже из математического анализа. Это значит, что функция растет строго медленнее другой, т.е. $f(n) = o(g(n))$ для любой константы c найдется такое N_0 , что для любого $n >$

N_0 выполняется $f(n) < c \times g(n)$. Иногда это бывает полезно для отбрасывания из рассмотрения определённых кусков алгоритма, но в целом используется реже.

Также из матанализа при оценке сложности алгоритма используется функция Ω . Запись $T(n) = \Omega(g(n))$ означает, что количество действий, совершаемых алгоритмом, не меньше, чем $c \times g(n)$ (всё как в O , меняется только знак). Эта оценка бывает полезна, когда нужно доказать, что алгоритма быстрее не существует.

Бывает самая точная оценка сложности алгоритма: Θ . Если мы сказали, что время выполнения алгоритма $T(n) = \Theta(g(n))$, то это означает, что найдётся такие константы c_1 и c_2 и число N_0 , что для любого $n > N_0$ будет выполнено $c_1 \times g(n) \leq T(n) \leq c_2 \times g(n)$. Т.е. $T(n) = O(g(n))$ и одновременно с этим $T(n) = \Omega(g(n))$. А это значит, что алгоритм работает ровно с заданной асимптотикой. К сожалению, не для всех алгоритмов можно однозначно определить Θ , потому что при одном вводе алгоритм будет работать быстро (и искусственно замедлять его не особого смысла), а при другом вводе алгоритм будет работать медленно и достигнет своего O . Например, сделайте в сортировке пузырьком досрочный выход из цикла, если не произошло ни одного обмена, и про этот алгоритм уже нельзя будет сказать, что его $T(n) = \Omega(n^2)$, если дать ему на вход уже отсортированный массив, то $T(n) = O(n)$, что противоречит утверждению про $T(n) = \Omega(N^2)$.

Поэтому Θ мы будем употреблять редко и только когда это важно. В остальных случаях будем ограничиваться самым лучшим O , которое мы смогли доказать — это будет оценкой сложности алгоритма в худшем случае, а худший случай обязательно наступит.

1.5 Амортизированная сложность

Амортизированная сложность (не ароматизированная, а от латинского "amortisatio", ослабление) — это, грубо говоря, средняя сложность выполнения операции.

Рассмотрим на следующем знакомом вам примере: добавление элемента в конец массива. На Python это метод `append`, на C++ — `push_back`. Разберемся, как это устроено. Операционная система по запросу программы может выделить определенную последовательную область памяти для хранения массива. Договоримся, что выделение массива длины n занимает $O(n)$ времени. Проблема в том, что при добавлении элемента к массиву длиной n нам потребуется выделить массив длиной $n + 1$. Казалось бы, мы можем попросить у операционной системы всего лишь один элемент непосредственно после нашего массива, но эта ячейка памяти может быть уже занята. Поэтому мы вынуждены запрашивать новую память размером $n + 1$ и копировать туда все старое содержимое за $O(n)$. Если мы создадим пустой массив и будем последовательно добавлять туда n элементов, то сложность всего этого процесса составит $O(n^2)$, что неприемлемо медленно в большинстве случаев.

Чтобы избавиться от этой проблемы человечество придумало следующую идею: выделим изначально небольшой массив, длиной в 1 элемент. Назовем текущий размер массива *capacity*, вместимость. Текущее количество реально присутствующих в массиве элементов назовем *size*, размер. То есть пока $size < capacity$ мы можем спокойно добавлять элементы в конец массива, т.к. под них есть свободное место. Когда $size$ станет равен $capacity$, мы увеличим $capacity$ вдвое, выделим новую память и скопируем старое содержимое массива в новый. Т.е. когда заполнится 1 элемент, то выделим массив размером 2, когда заполнится 2 — выделим 4, затем 8, 16, 32 и т.д.

При этом может оказаться, что size примерно в два раза меньше capacity — это произойдет сразу после выделения вдвое большего массива. То есть, половина выделенных ячеек памяти зарезервирована, но не используется. Такова плата за скорость работы, которую мы сейчас и оценим.

Чтобы оценить сложность такого алгоритма выделения памяти, нам нужно оценить количество перевыделений памяти и затраты при одном перевыделении памяти. Пусть, для удобства, $n = 2^k$. Тогда нам понадобится k выделений новой памяти под массив, т.к. сначала будет создан массив размера 1, затем 2, затем 4, и т.д. до 2^k . Выразим количество перевыделений памяти через n : т.е. $n = 2^k$, то $k = \log_2 n$. Поскольку логарифмы по разным основаниям отличаются друг от друга в константу раз, то в оценке сложности используют просто запись $k = \log n$, подразумевая основание 2, но, вообще говоря, оно может быть любым и это не повлияет на асимптотическую сложность.

Итак, мы определили количество перевыделений памяти, оно равно $\log n$. Каждое перевыделение занимает не более n операций (больше чем у нас есть элементов мы точно не скопируем), а значит общая сложность добавления n элементов в динамически расширяемый массив можно оценить как $O(n \log n)$. Амортизированная (в нашем случае средняя) оценка сложность выполнения одной операции добавления одного элемента в динамически расширяемый массив составит $O(\log N)$. Это правильная оценка, но не оптимальная. Давайте посчитаем внимательнее.

При первом перевыделении мы копируем 1 элемент, при втором — 2, при третьем — 4, и т.д. Таким образом, всего будет скопировано $2^0 + 2^1 + 2^2 + \dots + 2^k = 2^{k+1} - 1$. Т.е. суммарное количество копирований $T(n) = O(2^{k+1}) = O(2 \times 2^k) = O(2^k) = O(n)$. Таким образом, оказалось, что мы реализовали динамически расширяемый массив, который на добавление n элементов тратит $O(n)$ времени, а значит амортизированная (средняя) сложность добавления элемента составляет $O(1)$.

Получается, что динамически расширяемый массив асимптотически работает также быстро, как массив, для которого мы знаем размер заранее. При аккуратном подсчете мы даже можем прикинуть отличие константы, оно будет примерно равно двум, т.е. в реальности такой массив должен работать примерно вдвое медленнее того, размер которого известен заранее. Вполне приемлемо.

Таже мы с вами, фактически, придумали и оценили сложность операций в структуре данных — динамически расширяемом массиве. Это первый пример структуры данных.

Вернемся к амортизированной сложности операции. Несмотря на то, что в среднем сложность добавления элемента в массив занимает $O(1)$, в худшем случае, который происходит в тот момент, когда происходит расширение массива, сложность такой операции составит $O(n)$, где n — это длина массива. В некоторых ситуациях это может быть неприемлемо.

Системами реального времени называются такие системы, в которых реакция системы последует в течение гарантированного промежутка времени. Например, в беспилотном автомобиле требуется быстрый отклик на дорожную ситуацию. Допустим, мы использовали в беспилотном автомобиле динамически расширяемый массив для хранения каких-нибудь логов. Пользователь совершал длинную поездку и массив получился большим. В этот момент перед автомобилем внезапно возникла стена и надо бы резко нажать на тормоз, но наш динамически расширяемый массив с логом именно в этот момент решил расшириться и ушел в копирование гигантского количества элементов. Результатом будет трагедия, поэтому к сложным ответственным задачам людей, не знающих алгоритмов и структур данных не подпускают, а в системах реального времени

используют алгоритмы и структуры данных с гарантированной, а не амортизированной сложностью. В нашем примере это может быть кольцевой буфер или связный список.

1.6 Пространственная сложность

Аналогично временной сложности алгоритма мы можем оценить пространственную сложность — потребление памяти. Вместо элементарных операций мы будем считать количество ячеек памяти, которое использует программы.

Очевидно что пространственная сложность не может быть больше временной, т.к. ячейки памяти надо чем-то заполнить, а на это уходит одна операция. В учебных задачах, обычно, на работу программы выделяется одна или несколько секунд и за это время достаточно сложно занять всю выделенную память. Однако в реальном мире программы могут работать в течение минут, часов или даже дней и потребление памяти становится важной проблемой. Если для медленного алгоритма достаточно подождать чуть дольше, то память необходимо покупать, а это дорого или вовсе невозможно.

Также существует огромный класс достаточно быстрых устройств с минимальным объемом памяти — микроконтроллеры. Они стоят очень дешево, их процессоры имеют частоты в десятки и сотни мегагерц (а значит могут исполнять миллионы и десятки миллионов операций в секунду), но их память ограничена килобайтами, то есть тысячами ячеек памяти. Для них пространственно эффективные алгоритмы очень важны.

Часто оказывается, что мы можем пожертвовать скоростью работы алгоритма и получить меньшее потребление памяти. Обычно это происходит за счет того, что подсчитанные значения не запоминаются, а пересчитываются заново.

Как один из возможных вариантов более медленной, но менее требовательной к памяти структуры данных можно рассмотреть динамический массив, который каждый раз расширяется на один элемент. Сложность добавления n элементов составит $O(n)$ вместо $O(1)$, зато лишняя память потребляться не будет.

1.7 Тестирующая система

Придумать алгоритм — это только половина дела, его необходимо также реализовать. Для проверки ваших решений будет использоваться автоматизированная проверяющая система. Вы должны сдать на проверку программа, она будет запущена на наборе заранее подготовленных тестов и ответ будет сверяться с правильным.

При реализации алгоритма можно примерно рассчитывать на следующую производительность: на языке Python за одну секунду будут выполняться от десятков до сотен миллионов операций (от 10^7 до 10^8), а на C++ — от сотен миллионов до миллиарда операций (от 10^8 до 10^9). Таким образом, вы можете оценить асимптотическую сложность придуманного алгоритма, подставить в оценку ограничение из условия и проверить, успеет ли программы выполнить все необходимые действия за отведенный лимит времени. Если программа заведомо не успеет, то либо вы ошиблись с оценкой сложности в большую сторону, либо надо придумывать более эффективный алгоритм. При этом скорость работы программы никак не влияет на оценку, если программа укладывается в отведенный лимит. Поэтому в некоторых задачах может быть достаточно реализовать не самый эффективный алгоритм. Если он проще в реализации и укладывается в лимиты, то это нормально.

Вы можете получить один из следующих вердиктов:

- *OK* — программа прошла все тесты
- *WA* — программа выдала неправильный ответ на одном из тестов. Либо ваш алгоритм неверный, либо допущена программистская ошибка, либо не обработан один из крайних случаев (особенно, если ошибка произошла на тесте с небольшим номером)
- *PE* — вывод программы не соответствует формату. Например, программа не вывела ничего или вывела строку вместо числа
- *TL* — превышен лимит времени. При этом программа принудительно останавливается. Это возможно, если вы реализовали слишком медленный алгоритм, реализовали правильный алгоритм слишком неэффективно или в вашей программе есть вечный цикл
- *ML* — программа превысила лимит по памяти. Придумайте алгоритм, потребляющий меньше памяти
- *RE* — программа завершилась аварийно, например, в результате выхода за пределы массива, при делении на ноль. На Python программа завершается аварийно по многим поводам

При любом вердикте, кроме *OK* вам сообщается, на каком по номеру тесте программа сработала неправильно. Первые тесты — это тесты из условия, а следом за ними, обычно, идут небольшие тесты и краевые случаи, например, когда на вход подается пустая последовательность или последовательность из одного элемента. Придумывайте свои тесты, проверяйте свою программу, не забывайте проверять ее при минимальных возможных значениях переменных.

1.8 Самостоятельное тестирование

Содержимое теста, на котором программа не работает, вам не сообщается. Чтобы найти ошибку попробуйте придумать небольшие входные данные и ответы к ним. Проверьте на них свою программу. Если вы придумали много тестов и программа выдает правильные ответы, значит вы забыли про какой-то случай или неправильно поняли задачу. Перечитайте задачу и внимательно проверяйте краевые случаи.

Если ошибку так и не удалось найти, то вы можете воспользоваться методом, который называется «стресс-тестирование». В больших проектах этим термином называют тестирование с большой нагрузкой, а при решении алгоритмических задач этот термин имеет другой смысл. Вы реализуете простое, но неэффективное решение и оборачиваете его в функцию с названием "slow". Вы реализуете быстрое решение, которое работает неправильно и оборачиваете его в функцию "fast". Затем вы генерируете небольшие случайные тесты и запускаете на них оба решения до тех пор, пока не окажется, что ответы различаются. После этого у вас есть тест и правильный ответ к нему, и вы можете осуществить отладку. Не стоит генерировать большие тесты, т.к. на них сложно отлаживать свою программу.

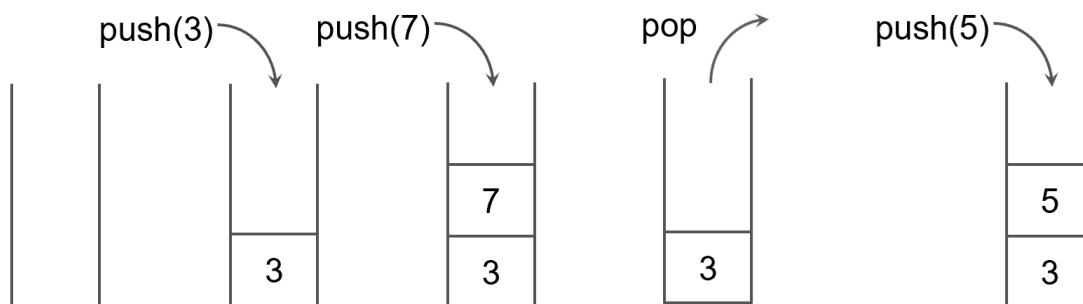
Лекция 2

Стеки, очереди и деки

2.1 Стек

Стек — это структура данных которая работает по принципу «последний пришел — первый вышел» (LIFO — Last In First Out). Можно представить себе стопку тяжелых книг, где вы можете положить книгу сверху, забрать книгу сверху (из середины нельзя — все обрушится) и посмотреть тоже только на книгу сверху. Также вы можете посчитать количество книг в стопке.

Таким образом, основных операций у стека всего четыре: **push(x)** — добавить элемент в стек, **pop()** — извлечь элемент из стека, **top()** или **peek()** — узнать значение элемента на вершине стека, **size()** — узнать размер стека. Пример работы со стеком на рисунке:



Для реализации стека подойдет обычный динамический массив: операции будут выполняться амортизированно за $O(1)$. В `list` из `python` есть все необходимые операции: добавление элемента делается с помощью `append`, удаление — `pop`, просмотр последнего элемента через обращение к индексу `[-1]`, а размер узнается стандартной функцией `len`.

В системах реального времени амортизированная сложность недопустима, поэтому придётся либо использовать стек с ограниченной вместимостью (при этом количество выполняемых над ним операций может быть намного больше, вместимость — это максимальное количество элементов, которые могут находиться в стеке одновременно), либо использовать связные списки. Связные списки (когда каждый элемент имеет указатель на соседей, или хотя бы одного соседа), позволяют реализовать все операции стека за $O(1)$ в худшем случае, но константа по времени и памяти будет достаточно большой и на практике такой метод реализации стека практически не используют.

На первый взгляд структура данных, которая имеет доступ только к «верхнему» элементу и прячет остальные элементы от пользователя, может показаться довольно

бесполезной, но стек имеет большое количество применений. Ограниченное количество операций позволяет создавать гарантии выполнения некоторых свойств данных, которые будут полезны при решении многих задач.

2.2 Правильная скобочная последовательность

Правильная скобочная последовательность (ПСП) — это такая последовательность из открывающих и закрывающих скобок, в которой каждая открывающая скобка имеет соответствующую ей закрывающую и вложенность корректна. Более формально, назовем ПСП следующие последовательности из круглых скобок:

- пустая последовательность
- если S — ПСП, то (S) — ПСП. Т.е. ПСП, окруженная скобками, тоже ПСП
- если S_1 и S_2 — ПСП, то S_1S_2 (две ПСП записанные подряд) — ПСП

Для проверки правильности скобочной последовательности из одного типа скобок мы можем использовать одну числовую переменную, в которой будет храниться разность между количеством открытых и закрытых скобок в каждый момент времени. Назовем эту переменную балансом. Если нам встретиться открывающая скобка, то прибавим к балансу 1, а если закрывающая — вычтем 1. Если в какой-то момент времени баланс стал отрицательным, то у нас нашлась закрывающая скобка, для которой нет соответствующей открывающей, а значит последовательность не является ПСП. Если после обработки всей последовательности баланс не равен нулю, то не все открывающие скобки закрылись, а значит последовательность не является ПСП.

Несколько примеров обработки последовательности с одним видом скобок. Во второй строке показан баланс после обработки соответствующей скобки:

Правильная последовательность, в конце 0, баланс никогда не был отрицательным:

(	(	)	(	)	)
1	2	1	2	1	0

Неправильная последовательность, баланс стал отрицательным в процессе работы:

(	(	)	)	)	(
1	2	1	0	-1	

Неправильная последовательность, баланс не равен нулю в конце:

(	(	)	(	(	)
1	2	1	2	3	2

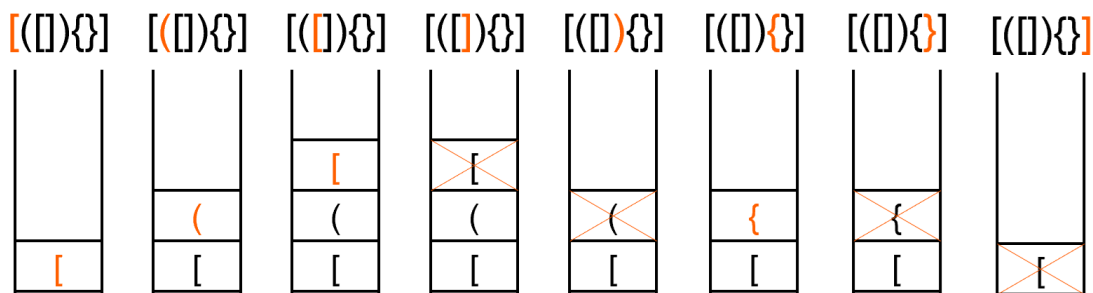
Теперь сделаем последовательность из нескольких видов скобок: круглых, квадратных и фигурных. Определение ПСП станет таким:

- пустая последовательность
- если S — ПСП, то (S) , $[S]$, $\{S\}$ — ПСП. Т.е. ПСП, окруженная скобками, тоже ПСП
- если S_1 и S_2 — ПСП, то S_1S_2 (две ПСП записанные подряд) — ПСП

Первым делом мы можем попробовать реализовать идею, аналогичную проверке скобочной последовательности на правильности с одним типом скобок. Заведем три счетчика, по одному для каждого типа скобок, и проверим правильность скобочной последовательности аналогично последовательности с одним типом скобок. Часть неправильных скобочных последовательностей будет определяться с помощью такого подхода, но существуют последовательности, которые будут ложно определены как правильные. Например, $[(\)]$. Ни один из счетчиков не стал отрицательным в процессе и в конце все счетчики равны нулю, однако последовательности неправильная.

Чтобы определять правильность скобочной последовательности с несколькими видами скобок воспользуемся стеком. Открывающие скобки будут добавлять в стек, а когда в последовательности встретится закрывающая скобка, проверим, что стек не пуст и на вершине стека находится соответствующая закрывающей открывающая скобка. Если стек пуст, то для закрывающей скобки не нашлось соответствующей ей открывающей, а если на вершине лежит открывающая скобка другого типа, то нарушилось правило о корректной вложенности скобок.

Пример обработки последовательности с несколькими типами скобок показан на рисунке:



Сложность такого алгоритма составит $O(N)$. Для удобства реализации бывает полезно завести словарь, в котором каждой закрывающей скобке соответствует открывающая такого типа — это избавит от большого количества ветвей в `if`.

Идея проверки скобочной последовательности на правильности используется, например, в редакторах кода для подсветки незакрытых скобок или лишних закрывающих скобок. Также эта идея может быть использована для проверки корректности HTML или XML-документов. Например, HTML `<a><b>text</a></b>` вообще говоря некорректен, хотя и умеет обрабатываться современными браузерами. Чтобы проверить корректность HTML мы можем интерпретировать открывающий тег как открывающую скобку, закрывающий — как закрывающую, а сам тег будет определять тип скобки. Корректный HTML будет отображаться браузерами быстро и однозначно, а сломанный XML вообще не сможет быть прочитан во многих случаях.

Кроме того, сама идея вложенности друг в друга отрезков (в данном случае отрезков от открывающей до соответствующей ей закрывающей скобки), будет часто встречаться нам в дальнейшем при изучении алгоритмов.

2.3 Инфиксная и постфиксная запись

При работе с арифметическими выражениями мы привыкли иметь дело с инфиксной записью, когда операция записывается между операндами. Например, в выражении $2 + 3$: $+$ — операция, а 2 и 3 — операнды. Также в привычных нам выражениях встречается

префиксная запись, когда сначала пишется операция, а затем операнды. Например, $\sin(x)$, где $\sin$ — операция, а x — операнд.

Если вложенность скобок в инфиксной записи будет очень большой, то легко запутаться и ошибиться в порядке вычислений, да и просто определение глубины вложенности скобок и порядке действий — довольно сложная операция. Поэтому инфиксную запись лучше преобразовать к постфиксной, когда сначала идут операнды, а затем операции. Также постфиксную запись называют «обратной польской записью» в честь польского математика Якова Лукасевича, который изучал ее свойства в 1920 году.

Для инфиксной записи:

$$6 + 3 \times (1 + 4 \times 5) \times 2$$

Постфиксная запись будет выглядеть как:

$$6\ 3\ 1\ 4\ 5\ \times\ +\ \times\ 2\ \times\ +$$

Посчитать значение выражения в постфиксной записи очень просто: мы будем последовательно складывать операнды в стек, а когда встретим операцию, то извлечем два операнда из стека, произведем операцию и положим ее результат в стек. При этом если операция не коммутативна (например, вычитание или деление) нам важно соблюдать порядок операндов: первый операнд — это предпоследний элемент в стеке, а второй — последний.

Для нашего выражения процесс вычисления выражения будет выглядеть так:

Текущий элемент	Стек до операции	Стек после операции
6 3 1 4 5 × + × 2 × +		6
6 3 1 4 5 × + × 2 × +	6	6, 3
6 3 1 4 5 × + × 2 × +	6, 3	6, 3, 1
6 3 1 4 5 × + × 2 × +	6, 3, 1	6, 3, 1, 4
6 3 1 4 5 × + × 2 × +	6, 3, 1, 4	6, 3, 1, 4, 5
6 3 1 4 5 × + × 2 × +	6, 3, 1, 4, 5	6, 3, 1, 4 × 5 = 20
6 3 1 4 5 × + × 2 × +	6, 3, 1, 20	6, 3, 1 + 20 = 21
6 3 1 4 5 × + × 2 × +	6, 3, 21	6, 3 × 21 = 63
6 3 1 4 5 × + × 2 × +	6, 63	6, 63, 2
6 3 1 4 5 × + × 2 × +	6, 63, 2	6, 63 × 2 = 126
6 3 1 4 5 × + × 2 × +	6, 126	6 + 126 = 132

Единственное оставшееся число в стеке, 132, является результатом вычисления выражения. Как видно, это очень простой процесс. Сложность алгоритма составляет $O(N)$, где N — суммарное количество операций и операндов. Осталось научиться переводить инфиксную запись в постфиксную.

Для перевода из инфиксной записи в постфиксную нам потребуется массив, в котором будет накапливаться постфиксная запись, а также стек операций.

Будем придерживаться следующих правил:

- Операнд сразу попадает в ответ
- Операция выталкивает из стека в ответ все операции, с приоритетом больше либо равным приоритету операции и кладется в стек
- Открывающая скобка кладется в стек

- Закрывающая скобка выталкивает из стека в ответ все операции до тех пор, пока на вершине стека не будет открывающая скобка, которая также удаляется из стека (но не попадает в ответ)
- После обработки всей входной последовательности все операции из стека выталкиваются в ответ

Эти правила были предложены Дейкстрой в 1961 году и называются «алгоритм сортировочной станции» (Shunting Yard).

Применим эти правила к нашему выражению:

Текущий элемент	Стек до	Стек после	Ответ
<u>6</u> + 3 × (1 + 4 × 5) × 2			<u>6</u>
6 <u>+</u> 3 × (1 + 4 × 5) × 2		<u>+</u>	6
6 + <u>3</u> × (1 + 4 × 5) × 2	+	+	6 <u>3</u>
6 + 3 <u>×</u> (1 + 4 × 5) × 2	+	+, <u>×</u>	6 3
6 + 3 × (<u>1</u> + 4 × 5) × 2	+, ×	+, ×, <u>(</u>	6 3
6 + 3 × (<u>1</u> + 4 × 5) × 2	+, ×, (	+, ×, (	6 3 <u>1</u>
6 + 3 × (1 <u>+</u> 4 × 5) × 2	+, ×, (	+, ×, (, <u>+</u>	6 3 1
6 + 3 × (1 + <u>4</u> × 5) × 2	+, ×, (, +	+, ×, (, +	6 3 1 <u>4</u>
6 + 3 × (1 + 4 <u>×</u> 5) × 2	+, ×, (, +	+, ×, (, +, <u>×</u>	6 3 1 4
6 + 3 × (1 + 4 × <u>5</u>) × 2	+, ×, (, +, ×	+, ×, (, +, ×	6 3 1 4 <u>5</u>
6 + 3 × (1 + 4 × 5) <u>×</u> 2	+, ×, (, +, ×	+, ×	6 3 1 4 5 <u>×</u> +
6 + 3 × (1 + 4 × 5) <u>×</u> 2	+, <u>×</u>	+, <u>×</u>	6 3 1 4 5 × + <u>×</u>
6 + 3 × (1 + 4 × 5) × <u>2</u>	+, ×	+, ×	6 3 1 4 5 × + × <u>2</u>
6 + 3 × (1 + 4 × 5) × 2	<u>+, ×</u>		6 3 1 4 5 × + × 2 <u>×</u> +

Перевод из инфиксной записи в постфиксную также работает за $O(N)$.

Есть одна необычная операция в привычных нам инфиксных выражениях — унарный минус. Его начертание никак не отличается от бинарного минуса, но обрабатываться он должен особым образом. Самое простое что можно сделать — добавить нули перед унарным минусом. Действительно, унарный минус может следовать только в самом начале выражения или непосредственно после открывающей скобки. Достаточно проверить только эти места и, при необходимости, вставить перед унарным минусом число ноль, чтобы минус превратился в бинарный.

В алгоритм преобразования из инфиксной записи в постфиксную достаточно легко добавлять операции. Например, очень просто ввести возведение в степень — его приоритет будет выше, чем у умножений и делений. Также можно добавить многие другие операции, характерные для языков программирования, например, вызов метода, присваивание и т.д. и т.п. В результате аккуратной обработки всех операций и правильного определения их приоритетов мы можем сделать интерпретатор языка программирования.

Однако, введение новых операторов может потребовать разделения операций по ассоциативности. В нашей реализации, когда операция выталкивает в ответ все операции с тем же приоритетом, реализуется лево-ассоциативность, то есть одинаковые операции будут выполняться слева-направо (как они встречаются в инфиксной записи). При этом, например, возведение в степень является право-ассоциативной операцией. Для нее нужно выталкивать из стека только операции с приоритетом, строго большим, чем у нее, и не выталкивать операции с тем же приоритетом.

2.4 Монотонные стеки

Идея поддержки в стеке монотонно возрастающей или убывающей последовательности может быть полезна для решения множества задач. Например, мы хотим для каждого элемента найти индекс ближайшего справа меньшего элемента, а если такого нет — записать для этого элемента число N (размер массива).

Эту задачу можно решить с помощью монотонного стека. Иногда такие стеки также называют «стек рекордов». Будем обрабатывать последовательность чисел слева направо и хранить в стеке те элементы, для которых еще не найден меньший элемент. При обработке очередного элемента из последовательности мы посмотрим на вершину стека: если на вершине стека лежит большее число, то это означает, что для него мы еще не знали, каким числом оно ограничено, т.е. между текущим числом и числом, лежащим на вершине стека, не было ни одного числа, меньшего, чем число на вершине стека.

Это означает, что в этот момент мы понимаем, что текущее число и есть ближайшее меньшее справа для того, числа, которое лежит на вершине стека. Запоминаем ответ для числа с вершины стека (индекс текущей позиции) и удаляем число с вершины стека — теперь ответ для него известен. После этого стек либо опустеет, либо на его вершине будет другое число. Если оно больше текущего, то повторяем процесс. А если меньше, то это означает, что для числа на вершине стека мы пока не нашли ответ. После этого кладем текущее число на вершину стека, ведь ответ для него пока не известен. Таким образом, в стеке всегда будет храниться неубывающая последовательность. Число кладется в стек только после выталкивания всех больших чисел.

После того, как мы дойдем до конца последовательности, для всех оставшихся в стеке чисел мы можем записать ответ N . На английском этот подход называется Next Smaller Element (NSR).

Для удобства реализации в стеке лучше хранить пару из значения и индекса элемента. Проиллюстрируем на примере последовательности 7 2 4 5 3

Текущий индекс: **0**

Обрабатываемое число: **7** 2 4 5 3

Стек до: []

Стек после: [(**7**, **0**)]

Ответ: [-, -, -, -, -]

Комментарий: Число 7 никого не выталкивает из стека. В стек кладется пара из числа 7 и его индекса 0.

Текущий индекс: **1**

Обрабатываемое число: 7 **2** 4 5 3

Стек до: [(**7**, **0**)]

Стек после: [(**2**, **1**)]

Ответ: [**1**, -, -, -, -]

Комментарий: Число 2 меньше, чем число 7 на вершине стека. Для индекса числа 7 (это 0, второе число из пары, которая находится на вершине стека) запоминаем в качестве ответа текущий индекс. Стек опустел и туда попадает пара из текущего числа (2) и его индекса (1).

Текущий индекс: **2**

Обрабатываемое число: 7 2 **4** 5 3

Стек до: [(7, 0)]

Стек после: [(2, 1), (**4, 2**)]

Ответ: [1, -, -, -, -]

Комментарий: Число 4 больше, чем число 2 на вершине стека. Из стека ничего не выталкивается, пара 4 (число) и 2 (его индекс) кладется в стек.

Текущий индекс: **3**

Обрабатываемое число: 7 2 4 **5** 3

Стек до: [(2, 1), (4, 2)]

Стек после: [(2, 1), (4, 2), (**5, 3**)]

Ответ: [1, -, -, -, -]

Комментарий: Число 5 больше, чем число 4 на вершине стека. Из стека ничего не выталкивается, пара 5 (число) и 3 (его индекс) кладется в стек.

Текущий индекс: **4**

Обрабатываемое число: 7 2 4 5 **3**

Стек до: [(2, 1), (**4, 2**), (**5, 3**)]

Стек после: [(2, 1), (**3, 4**)]

Ответ: [1, -, **4, 4**, -]

Комментарий: Число 3 меньше, чем число 5 на вершине стека. Для индекса числа 5 (это 3, второе число из пары, которая находится на вершине стека) запоминаем в качестве ответа текущий индекс. Аналогично, 3 меньше 4, находящегося на вершине стека. Индекс числа 4 — 2 (второй элемент из пары). Для индекса 2 запоминаем, что число на этой позиции ограничено справа числом с текущим индексом (4). На вершине стека оказывается число 2, которое меньше текущего числа 3. Поэтому пара из текущего числа (3) и его индекса (4) кладется в стек.

Текущий индекс: **5**

Обрабатываемое число: 7 2 4 5 3

Стек до: [(**2, 1**), (**3, 4**)]

Стек после: []

Ответ: [1, **5**, 4, 4, **5**]

Комментарий: Мы обработали всю последовательность и для всех чисел (точнее, их индексов) записываем в ответ, что они ограничены числом с индексом N (длина массива). Это признак того, что для этих чисел нет меньших чисел справа.

Этот алгоритм работает за $O(N)$ — каждое число будет один раз положено в стек и один раз из него извлечено. При этом одна операция выполняется за $O(1)$ амортизированно, т.к. если очередное число окажется очень маленьким, а до него шла возрастающая последовательность, то количество извлечений из стека может быть очень большим, вплоть до $N - 1$.

С помощью этого алгоритма можно решить известную задачу о поиске прямоугольника максимальной площади. Сформулируем ее так: у нас есть несколько подряд стоящих домов с заданной высотой и шириной 1. Мы хотим разместить прямоугольный рекламный баннер максимальной площади на фасадах этих домов. Нижняя сторона

баннера должна быть параллельна земле, а весь баннер должен размещаться на фасадах домов.

У этой задачи существует очевидное решение за $O(N^3)$ — перебрать левый дом, с которого начинается размещение баннера, и правый дом, на котором баннер заканчивается. Третий вложенный цикл будет перебирать высоты всех домов между левым и правым и находить минимум. Таким образом, мы рассмотрим все возможные пары из левой и правой границы баннера и найдем максимальную высоту баннера для этих границ.

Это решение легко оптимизировать до $O(N^2)$ — будем перебирать все возможные левые границы, а перебор правой границы совместим с поиском текущего минимума между левой и правой границей. Т.е. сначала положим правую границу равной левой и будем постепенно двигать ее направо, запоминая минимальную высоту дома между левой границей и текущим положением правой границы. Это позволит избавиться от третьего цикла.

Существует и линейное решение этой задачи. Действительно, рассмотрим ответ: это прямоугольник, который мы не можем увеличить в высоту (его верхний край ограничен высотой минимального из домов, на которых размещен баннер), а также мы не можем увеличить его влево или вправо (если бы это было возможно, то мы бы увеличили и получили лучший ответ). То есть слева и справа от прямоугольника находятся дома с высотой, меньшей чем высота минимального дома, на которых размещен прямоугольник. Также баннер может быть ограничен концами последовательности, т.е. включать в себя первый или последний дом.

Таким образом, мы можем перебрать все прямоугольники, которые невозможно расширить вверх, влево и вправо, и выбрать среди них наилучший — это и будет ответом. Будем перебирать все дома в качестве ограничивающего прямоугольник сверху. Для каждого из них нам нужно найти ближайший более низкий дом слева и справа. А это можно сделать заранее, с помощью предложенного выше алгоритма. Поиск ближайшего меньшего слева можно организовать аналогично поиску ближайшего меньшего справа, просто запустим цикл справа-налево.

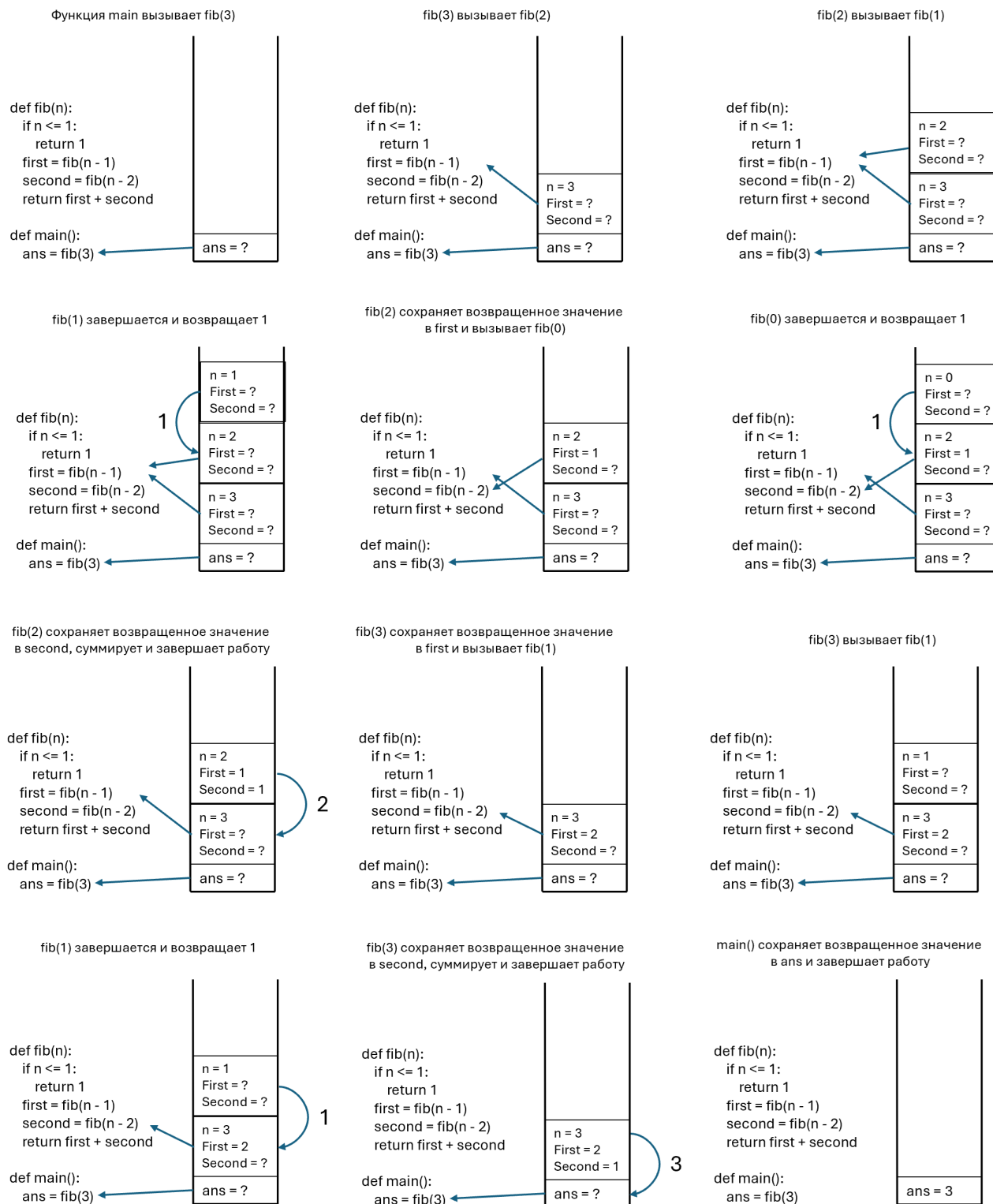
2.5 Стек вызовов

С помощью стека можно реализовать механизм вызова функций в императивных языках программирования, таких как Python или C++.

При вызове функции выполнение текущей функции приостанавливается до тех пор, пока не завершится работа вызванной функции. Это очередь удобно реализовывать в виде стека: для каждой запущенной функции будем выделять в стеке область памяти (называется кадр или фрейм), в котором будут храниться локальные переменные и параметры. При вызове другой функции мы будем запоминать во фрейме вызывающей функции то место, на котором остановилось выполнение функции. Когда вызванная функция завершит работу, то вызывающая функция продолжит свою работу с запомненного места. Когда функция завершает работу — ее кадр будет удален с вершины стека. Таким образом, всегда выполняется та функция, которая находится на вершине стека. Наверняка вы видели трассировку стека вызова функций в случае, если программа завершилась аварийно. Это и есть стек вызова функций.

Рассмотрим пример содержимого стека вызовов для рекурсивной функции подсчета

числа Фибоначчи:



Таким образом оказывается, что никакой магии в рекурсии нет и она может быть заменена стеком. Однако механизм продолжения работы функции с определенного места не очень просто реализуется во многих языках программирования. Поэтому рекурсия остается удобным механизмом.

2.6 Очередь

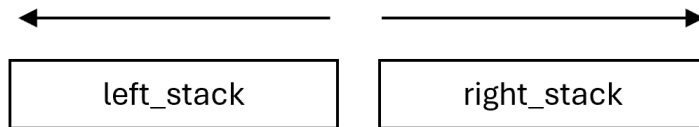
Все хорошо знакомы с понятием очереди: у нее есть конец, куда попадают новые элементы, а также начало, откуда элементы извлекаются. Очереди широко применяются в различных системах, например, при работе с сетью или организации различных буферов. По-английски очередь называется *queue* (читается как «кью»). У очереди есть операции добавления элемента в конец (*push*), удаления элемента из начала очереди (*pop_front*), просмотра значения элемента из начала очереди (*front*), а также можно узнать размер очереди.

В традиционной очереди, например, к кассе в магазине, при удалении элемента (клиент обслужен на кассе) каждый элемент должен подвинуться вперед. Аналогом настоящей очереди может выступать динамический массив, в котором начало очереди совпадает с началом массива, вот только элементы не будут двигаться сами, их нужно будет переставлять по одному. В такой реализации удаление элемента из начала очереди осуществляется за $O(N)$, что очень медленно.

Если проводить аналогию с очередью в магазине, то мы могли поступить следующим образом: становясь в очередь человек не двигается со своего места, а вместо этого двигается касса. Например, это можно реализовать по-прежнему с помощью динамического массива, где добавление элемента в конец происходит с помощью *append*, а для реализации «движущейся кассы» мы используем специальную переменную *begin*, которая будет означать индекс начала очереди. При удалении элемента достаточно увеличить этот индекс на единицу. В таком случае обе операции добавления и удаления будут иметь асимптотику $O(1)$, причем удаление будет работать строго за $O(1)$, а добавление — амортизировано. Основная проблема такой реализации состоит в том, что в начале массива будут копиться удаленные из очереди элементы. Проводя аналогию с магазином: касса уедет очень далеко от своего первоначального положения и реально будет использоваться небольшой участок магазина, в то время как большая часть будет пустовать. От этой проблемы можно избавиться, время от времени передвигая кассу к своему начальному положению (людям в этот момент придется подойти к кассе). При программировании это будет выглядеть, например, так: когда количество неиспользуемых элементов (т.е. *begin*) станет равен половине *capacity* массива, то мы копируем все содержимое очереди в начало массива, и устанавливаем *begin* на ноль. При этом *size* массива (т.е. конец очереди) должен показывать на реальный конец очереди.

В случае, когда мы не можем позволить себе амортизированную сложность у нас есть два возможных варианта действий: использовать связный список или его аналоги с лучшей константой или, в случае, если нам известно, что одновременно в очереди может находиться не более K элементов и это число сильно меньше N , общего количества операций с очередью, то мы можем воспользоваться кольцевым буфером. Аналогия кольцевого буфера в магазине — круг по которому движется касса. Люди встают на первое свободное место и стоят там в ожидании приезда кассы. Реализация кольцевого буфера возможна на обычном массиве, для этого достаточно брать числа *begin* и *size* (индексы начала и конца очереди) по модулю K . Кольцевые буферы широко используются во многих алгоритмах для экономии памяти и избавления от необходимости копирования данных.

Еще один любопытный способ реализовать очередь — это использовать два стека. Первый стек будет выступать началом очереди, а второй — концом.



При добавлении элемента в очередь мы будем добавлять его в правый стек, а при удалении — забирать из левого стека. Со временем левый стек опустеет. В этот момент мы можем перенести по одному все элементы из правого стека в левый. Тогда тот элемент который был на вершине правого стека станет самым глубоким элементом в левом, а тот, который был самым глубоким в правом стеке (он был добавлен раньше других элементов правого стека) окажется на вершине левого стека и будет извлечен первым. Такой подход также делает сложность амортизированной, но позволяет пользоваться стандартными структурами, не реализуя реаллокацию массива вручную. В Python есть готовый класс `collections.deque`, с помощью которого можно реализовать очередь.

2.7 Дек

Дек — это двусторонняя очередь, *double-ended queue*, *deque*.

В него можно добавлять и удалять элементы и с левого, и с правого конца. Операции для левого конца называются `push_front` и `pop_front`, а для правого — `push_back` и `pop_back`. Его также можно организовать на динамическом массиве с амортизированной сложностью, либо на связном списке, либо на кольцевом буфере, если мы знаем ограничение на максимальное количество элементов, одновременно находящихся в деке.

Рассмотрим одно из возможных применений дека: очередь с серединой. Пусть нам нужно поддерживать структуру данных, которая может выполнять три операции: удаление элемента из начала, добавление элемента в конец и добавление элемента в середину. Количество элементов в очереди с серединой может быть четным ($2K$) или нечетным ($2K + 1$). Элемент в середину надо вставлять после K -го элемента независимо от четности.

Для решения этой задачи нам потребуется поддерживать два дека: для первой и второй половин очереди. Назовем их «левый дек» и «правый дек». Удаление элемента всегда будет происходить из начала левого дека. Добавление в конец — в конец правого дека. А добавление в середину — конец левого дека. После любой операции для корректной работы надо поддерживать *инвариант* (свойство, верное после любого шага алгоритма): если общее число элементов в структуре четное, то оба дека должны быть одинакового размера, а если нечетное — правый дек должен быть на один элемент длиннее. В случае, если это свойство нарушилось, его необходимо восстановить, переложив левый элемент из правого дека в конец левого или наоборот. Этот процесс называется балансировкой.

2.8 Минимум в окне

Рассмотрим следующую задачу: для каждых K подряд идущих элементов массива длины n необходимо выбрать минимальный элемент. Например, если $N = 7$, содержимое массива 2, 4, 3, 1, 5, 6, 4, а $K = 3$, то ответом будет массив 2, 1, 1, 1, 4 ($\min(2, 4, 3) = 2$, $\min(4, 3, 1) = 1$, $\min(3, 1, 5) = 1$, $\min(1, 5, 6) = 1$, $\min(5, 6, 4) = 4$).

Эту задачу можно решить наивным алгоритмом за $O(NK)$. Существует и более быстрое решение.

Будем поддерживать множество чисел, которые находятся в текущем окне и могут стать минимумом в каком-либо более правом окне. Сначала нам необходимо заполнить это множество для первых K элементов. Будем обрабатывать числа последовательно. При добавлении числа в это множество, мы можем удалить из множества все числа, большие добавленного: в этом окне они точно не будут минимумом (т.к. уже больше добавленного). А также они не смогут быть минимумами ни в одном из последующих окон: все последующие окна, содержащие числа, которые находились в множестве ранее, будут содержать и текущее число, которое меньше. Таким образом, мы проходим по первым K числам и при добавлении очередного числа удаляем все большие числа. После этой обработки мы должны выбрать наименьшее число из множества — это и будет ответ для первого окна длиной K .

Теперь нам необходимо научиться сдвигать окно на один вправо. При сдвиге окна один элемент добавляется, его мы будем обрабатывать аналогично первым K элементам — удалим из множества все большие элементы и добавим в множество его. Также нам надо научиться убирать из множества элемент, который перестал находиться в окне при сдвиге.

Для эффективной реализации такого множества мы можем использовать монотонный дек. Элементы в нем будут упорядочены по возрастанию, как в монотонном стеке, а свойство дека нужно для удаления из начала, в случае если из окна был удален минимальный элемент.

Рассмотрим процесс обработки последовательности 2, 4, 3, 1, 5, 6, 4. Сначала мы обработаем первые $K = 3$ числа.

- Добавляется число 2, состояние дека: 2
- Добавляется число 4, состояние дека: 2, 4
- Добавляется число 3, оно выталкивает все большие числа из правой части дека (число 4), а затем попадает в дек. Состояние дека: 2, 3

После этого шага мы выводим левый, наименьший элемент дека: 2. На всех последующих шагах у нас будет добавление и удаление одного элемента.

- Удаляется число 2. Поскольку оно находится в начале дека, то удаляем его. Состояние дека: 3
- Добавляется число 1, оно выталкивает все большие числа из правой части дека (число 3), а затем попадает в дек. Состояние дека: 1
- Удаляется число 4. Поскольку оно не находится в начале дека, то это означает, что оно уже было удалено из дека ранее каким-то меньшим числом, которое следовало после него. Состояние дека не изменилось: 1
- Добавляется число 5, оно не выталкивает никого и попадает в дек. Состояние дека: 1, 5
- Удаляется число 3. Оно не стоит в начале дека, поэтому состояние дека не изменилось: 1, 5

- Добавляется число 6, оно не выталкивает никого и попадает в дек. Состояние дека: 1, 5, 6
- Удаляется число 1. Поскольку оно находится в начале дека, то удаляем его. Состояние дека: 5, 6
- Добавляется число 5, оно выталкивает все большие числа из правой части дека (сначала 6, затем 5), а затем попадает в дек. Состояние дека: 4

Такой алгоритм позволяет находить минимум в окне за линейное от размера массива ($O(N)$). При этом окно не обязано быть фиксированного размера и добавление или удаление элементов может подчиняться другим правилам. С такими примерами мы познакомимся в одной из следующих тем.

Лекция 3

Квадратичные сортировки и сортировка слиянием

3.1 Сортировка пузырьком

Сортировка пузырьком (Bubble sort) является одним из простейших и известных способов сортировки. Идея заключается в том, что на каждом шаге сравниваются все пары соседних элементов и если эти элементы стоят в неправильном порядке, то они обмениваются местами. Обычно под сортировкой понимается упорядочивание элементов по неубыванию, поэтому неправильный порядок для элементов — это когда элемент с меньшим номером стоит раньше, чем элемент с большим номером. Т.е. если $a_i > a_{i+1}$, то элементы меняются местами.

Название сортировки связано с тем, что «легкие» (маленькие) элементы постепенно «всплывают» (перемещаются) к началу массива. Каждый элемент в сортировке пузырьком движется к концу массива, обмениваясь с меньшими элементами, до тех пор, пока не дойдет до большего элемента или до конца массива. Таким образом, за один шаг сортировки пузырьком, самый тяжелый элемент окажется в конце массива.

Рассмотрим код сортировки пузырьком:

```
def bubble_sort(a):
    n = len(a)
    for i in range(n):
        for j in range(n - 1):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
```

Этот код работает за $\Theta(N^2)$. Однако он совершает много лишних действий и его константа достаточно велика. Мы не учитываем тот факт, что за один шаг алгоритма самый большой элемент оказывается на своем месте. Кроме того, достаточно совершить $N - 1$ шаг внешнего цикла, т.к. единственный обрабатываемый на последнем шаге элемент уже стоит на своем месте. Если учесть это, то константа снизится примерно вдвое:

```
def bubble_sort(a):
    n = len(a)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if a[j] > a[j + 1]:
```

$$a[j], a[j + 1] = a[j + 1], a[j]$$

Также можно проверять, был ли совершен хотя бы один обмен во внутреннем цикле. Если ни одного обмена не было, то это означает, что массив уже отсортирован. В «среднем» случае это может дать значительное ускорение, однако в худшем случае мы совершим больше действий, т.к. эта проверка будет выполняться на каждом шаге цикла и не даст никакой выгоды. Худший случай для сортировки пузырьком — массив, отсортированный по убыванию.

Сортировка пузырьком использует только сравнение соседних элементов, что позволяет сортировать связные списки.

3.2 Сортировка вставками

При сортировке вставками (Insertion sort) мы будем поддерживать в начале массива уже отсортированную часть. То есть после K -го шага в начале массива будут храниться первые K элементов исходного массива в отсортированном порядке.

После $K - 1$ -го шага в начале массива будет храниться в отсортированном порядке $K - 1$ первый элемент. Возьмем элемент, который стоит на K -м месте, найдем в отсортированной части место, куда он должен быть вставлен, и вставим его, передвинув вправо все большие элементы. Поиск и перемещение элементов можно совместить, если осуществлять поиск места для вставки справа-налево:

```
def insertion_sort(a):
    n = len(a)
    for k in range(n):
        now = a[k]
        new_place = 0
        for j in range(k - 1, -1, -1):
            if a[j] > now:
                a[j + 1] = a[j]
            else:
                new_place = j + 1
                break
        a[new_place] = now
```

Такой подход позволяет уменьшить количество присваиваний по сравнению с сортировкой пузырьком. В сортировке пузырьком мы обмениваем пару элементов, что требует двух присваиваний. В сортировке вставками для сдвига элемента нам достаточно только одного присваивания. В современном компьютере присваивание достаточно медленная операция по сравнению с чтением данных.

Кроме того, в среднем случае сортировка вставками на каждом шаге будет перебирать не все элементы массива, а, примерно, половину. Это также даст снижение константы. А если массив уже полностью отсортирован, то сортировка вставками завершит свою работу за $O(N)$. Также заметное ускорение будет и на частично отсортированных массивах.

Поскольку сортировка вставками использует только последовательный доступ к элементам, она также может быть применена для сортировки связных списков.

3.3 Сортировка выбором

Еще одна квадратичная сортировка — это сортировка, которая на K -м шаге поддерживает в начале массива отсортированный набор из K минимальных чисел массива.

Реализуется это достаточно просто: необходимо найти минимальный элемент среди элементов с K -го по $N - 1$ и поменять его местами с K -м элементом.

```
def selection_sort(a):
    n = len(a)
    for k in range(n):
        min_index = k
        for j in range(k + 1, n):
            if a[j] < a[min_index]:
                min_index = j
        a[k], a[min_index] = a[min_index], a[k]
```

Эта сортировка совершит только $\Theta(N)$ присваиваний, что делает ее незаменимой на архитектурах, где присваивание стоит очень дорого по сравнению с чтением. Тем не менее, эта сортировка будет работать за $O(N^2)$ операций даже в лучшем случае.

3.4 Сравнение квадратичных сортировок

Перед тем как сравнить между собой квадратичные сортировки, введем важное понятие: устойчивость. Сортировка называется устойчивой, если она сохраняет взаимный порядок элементов с одинаковыми ключами сравнения. Например, если на уроки физкультуры школьникам требуется выстроиться по росту, Вася и Петя имеют одинаковой рост и до сортировки Вася стоял левее Пети, то устойчивая сортировка сохранит их взаимный порядок. Вася по-прежнему будет стоять левее Пети.

Некоторые сортировки можно сделать устойчивыми без использования дополнительной памяти, а некоторые — нет. При этом любую сортировку можно сделать неустойчивой, просто написав плохую реализацию. Также любую сортировку можно сделать устойчивой, используя $O(N)$ дополнительной памяти. В этой памяти мы будем для каждого элемента хранить его позицию в исходном массиве, и, в случае совпадения ключей сортировки, сравнивать по этому параметру.

Сортировка пузырьком является устойчивой, для этого достаточно не менять местами одинаковые элементы.

Сортировку вставками также можно сделать устойчивой, для этого достаточно находить позицию для вставки элемента со значением X так, чтобы она находилась после позиций, на которых уже стоят элементы с таким же значением. Наша реализация сортировки вставками является устойчивой.

Сортировка выбором не является устойчивой. Рассмотрим пример, в котором свойство устойчивости не выполняется, для этого пометим одинаковые числа разным количеством штрихов, чтобы отличать их друг от друга. Пусть исходный массив содержал следующие числа: $2'$, $2''$, 1 . На первом шаге сортировки мы найдем минимальный элемент (1) и поменяем его местами с элементом на нулевом месте, после чего массив примет вид 1 , $2''$, $2'$. На следующих шагах перестановок элементов не будет, так что это окончательное состояние массива. Эту сортировку можно сделать устойчивой, однако это увеличит количество присваиваний до $O(N^2)$.

Рассмотрим таблицу сравнения сортировок:

Свойство	Пузырьком	Вставками	Выбором
Худший случай	$O(N^2)$	$O(N^2)$	$O(N^2)$
Лучший случай	$O(N^2)$	$O(N)$	$O(N^2)$
Кол-во присваиваний	$O(N^2)$	$O(N^2)$	$O(N)$
Устойчивая	Да	Да	Нет

При этом сортировка пузырьком обладает важным свойством: обмены происходят только между соседними элементами, что может быть полезно при некоторых дополнительных ограничениях.

В случае, если нам нужно минимизировать количество присваиваний и нам не важна устойчивость, можно использовать сортировку выбором.

В реальности же из квадратичных сортировок используется сортировка вставками, т.к. он отлично работает на почти отсортированных данных, которые достаточно часто возникают при решении реальных задач.

3.5 Сложность сортировок

Квадратичные сортировки пишутся очень просто, однако их сложность слишком велика для использования для длинных массивов. Обычно они используются только в случае, если нужно отсортировать небольшое количество элементов, например, 20 или менее.

Для длинных последовательностей существуют более эффективные сортировки. Нам необходимо понять за какую наименьшую асимптотическую сложность возможно отсортировать последовательность из N элементов.

Для такой теоретической оценки нам нужно понять, что такое задача сортировки. Для простоты рассмотрим случай, когда все элементы в последовательности различны. Тогда наш массив можно интерпретировать как перестановку из N элементов. Всего существует $N!$ таких перестановок и нам необходимо понять одну конкретную перестановку, которая позволит переставить элементы и сделать массив отсортированным.

Пусть мы можем задавать вопросы с ответами «да» или «нет» и получать на них ответ за $O(1)$. Например, таким вопросом может быть сравнение элементов последовательности с индексами i и j , но, вообще говоря, вопросы могут быть и другими. Чтобы угадать один из $N!$ вариантов нам необходимо задать $\log_2(N!)$ таких вопросов. Оценим, чему равно $\log N!$.

$$\log N! = \log(1 \times 2 \times 3 \times \cdots \times N) = \log 1 + \log 2 + \log 3 + \cdots + \log N$$

Эту сумму легко оценить сверху — достаточно взять самое большое слагаемое и заменить все N слагаемых на него. Таким образом, легко получить неравенство:

$$\log N! \leq N \log N$$

Теперь оценим эту сумму снизу. Рассмотрим слагаемые, больше либо равные $\log N/2$. Количество таких слагаемых примерно равно $N/2$ (при чётном N в точности равно) и каждое из этих слагаемых не меньше $\log N/2$. Таким образом, можно сделать оценку:

$$\log N! \geq N/2 \log N/2$$

Мы ограничили функцию сверху и снизу одной и той же функцией $X \log X$ с константами $1/2$ и 1 . Таким образом, мы можем утверждать, что:

$$\log N! = \Theta(N \log N)$$

А это значит, что нельзя написать сортировку, основанную на вопросах с двумя вариантами ответа, быстрее чем за $O(N \log N)$. Также это значит, что, теоретически, такие сортировки существуют.

3.6 Слияние отсортированных последовательностей

Рассмотрим следующую задачу: нам дано две отсортированных последовательности, необходимо получить из них одну. Эта задача решается достаточно просто. Будем поддерживать указатели на текущий элемент в каждой из последовательностей. Сначала указатели показывают на нулевой элемент в каждой из последовательностей. На каждом шаге в результирующую последовательность записывается меньший из двух элементов, на которые показывают указатели, и, соответствующий ему указатель, сдвигается. Если одна из отсортированных последовательностей закончилась, то нужно просто скопировать оставшиеся элементы из второй последовательности. Рассмотрим пример:

Первая	Вторая	Итог	Комментарий
2 , 2, 3	1 , 3, 5	1	Взять 1 из второй
2 , 2, 3	1, 3 , 5	2	Взять 2 из первой
2, 2 , 3	1, 3 , 5	2	Взять 2 из первой
2, 2, 3	1, 3 , 5	3	Взять 3 из первой
2, 2, 3	1, 3 , 5	3	Взять 3 из второй
2, 2, 3	1, 3, 5	5	Взять 5 из второй

Таблица 3.1: Слияние двух отсортированных последовательностей: $[2, 2, 3]$ и $[1, 3, 5]$

Обратите внимание, что в ситуации, когда указатели показывают на одинаковые элементы, мы берем элемент из первой последовательности. Вообще говоря, можно брать элемент из любой последовательности, но в будущем нам пригодится это свойство.

Также важно правильно оценить временную и пространственную сложность этого алгоритма слияния. Сложность составляет $O(N)$, где N — суммарная длина сливаемых последовательностей. Кроме того, для хранения слито последовательности нам потребуется дополнительный массив, его длина также составит N .

Самый простой способ реализации слияния состоит в следующем: будем N раз выбирать очередной элемент. При этом сформулируем условие, при котором мы берем элемент из первой последовательности:

- Первая последовательность не пуста
- Вторая последовательность пуста или текущий элемент из первой последовательности меньше, чем текущий элемент из второй последовательности

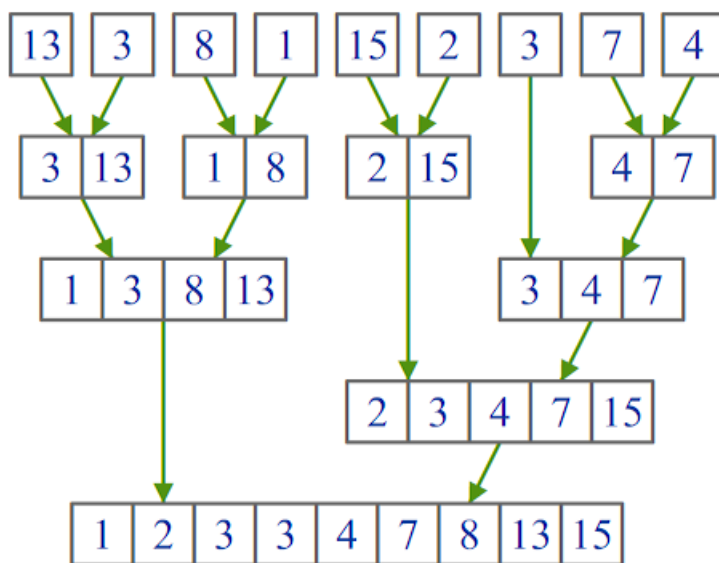
Во всех остальных случаях будем брать элемент из второй последовательности.

3.7 Сортировка слиянием

На основе функции слияния можно реализовать сортировку слиянием. Для этого используется принцип «разделяй и властвуй»: разобьем исходную последовательность из N элементов на две половины, каждую из которых отсортируем с помощью рекурсивной функции. А затем сольем две отсортированные последовательности. Схематично функцию можно представить так:

```
def merge_sort(a):
    n = len(a)
    if n == 1:
        return a
    return merge(merge_sort(a[: (n + 1) // 2]), merge_sort(a[(n + 1) // 2:]))
```

Процесс сортировки массива с помощью такой версии сортировки слиянием показан на рисунке:

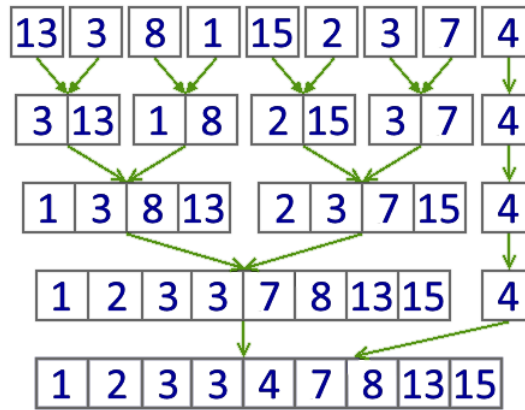


Сложность такой сортировки составит $O(N \log N)$. У нас есть $\log N$ уровней (т.к. каждый раз длина обрабатываемой последовательности уменьшается вдвое) и на каждом уровне каждый элемент массива будет обработан с помощью merge, суммарно это займет $O(N)$ времени для уровня.

Если в случае равенства элементов выбрать элемент из левой половины, то сортировка слиянием будет устойчивой.

Вызов рекурсивной функции достаточно долгая операция, поэтому для уменьшения константы можно использовать нерекурсивный вариант сортировки слиянием. Для этого разобьем элементы на пары и упорядочим эти пары с помощью merge. Затем рассмотрим упорядоченные пары и сольем их в четверки и так далее. Такой способ требует аккуратной реализации при обработке краевых эффектов (когда длина правой половины может не быть степенью двойки), а также аккуратного использования дополнительной памяти. Например, здесь хорошо подойдет кольцевой буфер длины 2.

В случае нерекурсивной реализации сортировки слиянием, процесс будет выглядеть следующим образом:



3.8 Подсчет числа инверсий

Число инверсий в перестановке — это количество пар (i, j) , таких что $i < j$ и $a_i > a_j$. В массиве могут быть одинаковые элементы, однако мы можем подсчитывать число инверсий в массиве по тому же правилу.

Число инверсий легко подсчитать с помощью сортировки пузырьком: в процессе выполнения этой сортировки неправильно стоящие элементы в какой-то момент окажутся соседними и обменяются местами. Таким образом, число инверсий в массиве в точности равно количеству обменов в сортировке пузырьком. Однако такой способ работает за $O(N^2)$.

Рассмотрим более эффективный алгоритм с использованием идеи сортировки слиянием. Пусть наш массив обладает следующим свойством: он разбит на две равные по длине половины (левую и правую), каждая из которых упорядочена по неубыванию. Внутри каждой из половин инверсий нет, т.к. они упорядочены. Инверсии образуются только для пар, в которых меньший элемент взят из правой половины, а больший — из левой.

Такой подсчет довольно легко реализовать с помощью операции слияния. Каждый раз, когда мы берем элемент из правой половины, к количеству инверсий будет прибавляться количество больших чем он элементов из левой половины. При этом во время слияния указатель в левой половине как раз будет показывать на первый больший элемент. Таким образом, количество инверсий для этого элемента из правой половины будет равно разности длины левой половины с текущим значением указателя в левой половине. То есть, если длина левой половины равна k , i — индекс текущего элемента в левой половине, а j — в правой половине, то в случае $a[j] < a[i]$ к числу инверсий прибавляется $k - i$.

После такого слияния мы получим отсортированную последовательность и подсчитаем количество инверсий для двух упорядоченных половин. Чтобы получить число инверсий для произвольного массива нам надо реализовать сортировку слиянием и единственное отличие от обычной сортировки будет в функции слияния, которая при выборе элемента из правой половины будет прибавлять к счетчику инверсий количество еще не обработанных элементов из левой половины.

Рассмотрим пример подсчета числа инверсий для массива $[4, 2, 5, 3, 1]$. Пусть мы уже рекурсивно разделили массив на половины $[4, 2]$ и $[5, 3, 1]$ и отсортировали их. Теперь будем сливать половины $[2, 4]$ и $[1, 3, 5]$, подсчитывая инверсии.

Левая	Правая	Выбрано	Инверсии	Комментарий
2 , 4	1 , 3, 5	1	+2	(2, 4) образуют инверсии с 1
2 , 4	1, 3 , 5	2	0	Взять 2 из левой
2, 4	1, 3, 5	3	+1	(4) образует инверсию с 3
2, 4	1, 3, 5	4	0	Взять 4 из левой.
2, 4	1, 3, 5	5	0	Взять 5 из правой. Нет инверсий

Таблица 3.2: Слияние и подсчет инверсий для подмассивов $[2, 4]$ и $[1, 3, 5]$

В данном примере при слиянии добавилось $2 + 1 = 3$ инверсии. Необходимо учитывать инверсии внутри левой и правой половины, поэтому общее число инверсий считается как:

$$\text{Инверсии} = \text{Инверсии слева} + \text{Инверсии справа} + \text{Инверсии при слиянии}.$$

Для массива $[4, 2, 5, 3, 1]$:

- Инверсии в левой половине $[4, 2]$: 1.
- Инверсии в правой части $[5, 3, 1]$: 3.
- Инверсии при слиянии: 3.
- Итого: 7.

Алгоритм рекурсивно разбивает массив на половины, подсчитывает инверсии в каждой половине и суммирует их с инверсиями, возникшими при слиянии.

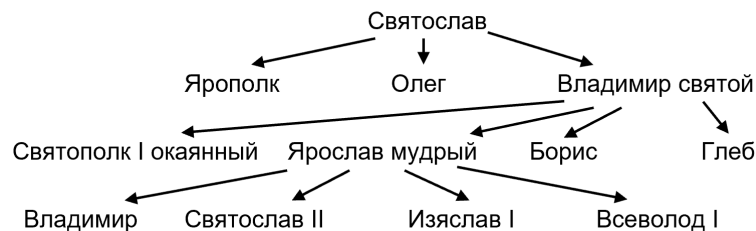
Сложность алгоритма остается той же, что и у обычной сортировки подсчетом: $O(n \log n)$.

Лекция 4

Куча, пирамидальная сортировка и бинарный поиск

4.1 Дерево

Рассмотрим структуру данных, под названием «дерево». Многие термины для описания деревьев используются из генеалогических деревьев, изучим их на примере фрагмента дерева династии Рюриковичей:



Каждый узел дерева в данном случае обозначается именем князя. Между узлом-родителем существует связь с узлами-детьми. У каждого узла, кроме корня, есть ровно один родитель. Обычно, для узла дерева запоминается список детей, а также, иногда, запоминается родительский узел. Используемые термины интуитивно понятны: «потомок», «предок», «родитель» (или «отец»), ребенок (или «сын»). Иногда возникают более экзотические, но все так же интуитивно понятные термины, такие как «дед», «дядя» и т.п.

Кроме этого, в деревьях используется терминология из ботаники. Например, основатель династии (Святослав) называется «корнем» дерева. При этом при рисовании дерева корень, обычно, оказывается наверху, что необычно для ботаники. Узлы, не имеющие потомков, называются «листья». Иногда используются и другие термины, например «ветвь». Они также интуитивно понятны.

Важный термин — «поддерево». Поддеревом имеет собственный корень и включает в себя всех потомков корня, не только непосредственно детей, но и внуков и т.д. Это тоже достаточно интуитивно понятный термин из генеалогических деревьев.

Тот факт, что поддерево также является деревом очень полезен при обработке деревьев с помощью рекурсивных функций. Часто рекурсивная функция вызывается для всех детей узла, обрабатывает возвращенные значения и, в свою очередь, возвращает значение родителю. При этом обработка деревьев чаще всего начинается с корня.

4.2 Куча

Куча (по-английски *heap*) — структура данных, поддерживающая операции для набора данных:

- Нахождение минимума
- Удаление минимума
- Добавление элемента

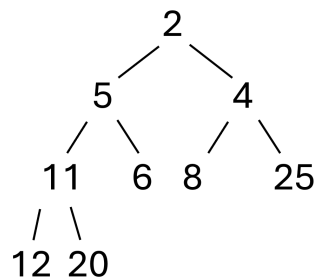
В некоторых источниках вместо неблагозвучного названия «куча» используют термин «пирамида», но в целом он не прижился.

Куча реализует абстрактную структуру данных «очередь с приоритетом». В такой очереди каждый элемент имеет собственный приоритет и раньше должен обрабатывать элемент с наименьшим приоритетом. Такого рода процессы часто происходят в реальном мире и при обработке данных, так что такая структура данных является довольно востребованной.

Куча минимумов представляет собой бинарное дерево, в котором каждый узел меньше своих детей. «Бинарное дерево» (или «двоичное дерево») означает, что каждый узел имеет не более двух детей, которых называют «левым» и «правым». Поскольку свойство кучи выполняется для всех узлов, то можно сказать, что значение в узле в куче минимумов меньше, чем у всех потомков. То есть значение в узле — это минимум в поддереве, корнем которого является этот узел.

В принципе, если в куче могут храниться одинаковые элементы, то условие можно изменить на «значение в каждом узле меньше либо равно, чем значение в детях». Также существует куча максимумов, в которой правило обратное: значение в каждом узле больше, чем значение в детях.

Чтобы добиться высокой скорости операций кучу заполняют плотно — все узлы, не являющиеся листьями, имеют двух детей, кроме, возможно, одного элемента. Приведем пример кучи:



Если в куче K уровней, то максимальное количество элементов в ней будет равно:

$$2^0 + 2^1 + 2^2 + \dots + 2^{K-1} = 2^K - 1$$

В худшем случае на последнем уровне будет находиться всего один элемент, поэтому минимальное количество элементов с K уровнями равно:

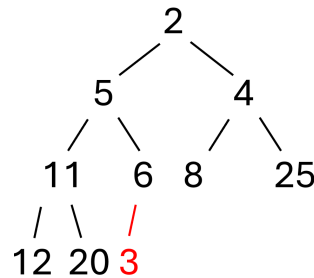
$$2^0 + 2^1 + 2^2 + \dots + 2^{K-2} + 1 = 2^{K-1}$$

Таким образом, количество элементов в куче $N = \Theta(2^K)$. Чаще нам будет нужно обратное соотношение $K = \Theta(\log N)$. То есть количество уровней в куче пропорционально логарифму от количества элементов в ней.

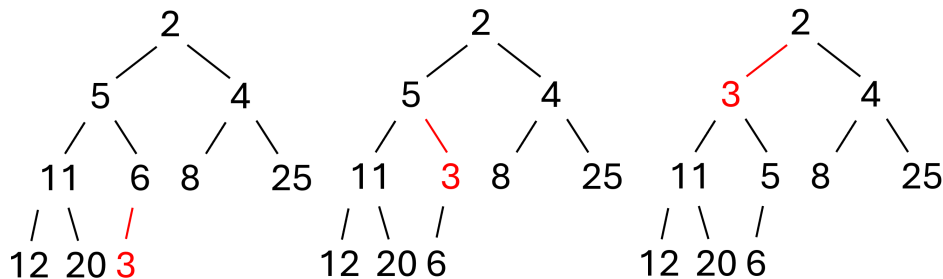
Научимся выполнять операции с кучей.

Самая простая операция — нахождение минимума. Минимум находится в корне кучи, его можно узнать за $O(1)$.

При добавлении элемента он вставляется на первое свободное место в нижнем ряду, а если нижний ряд занят, то создается новый ряд. Добавим к куче элемент 3:



В результате добавления этого элемента свойство кучи нарушилось — элемент 3 меньше своего отца 6. Поменяем их местами и продолжим обменивать новый элемент с отцом, пока это необходимо. Этот процесс называется «просеиванием вверх» (sift-up):



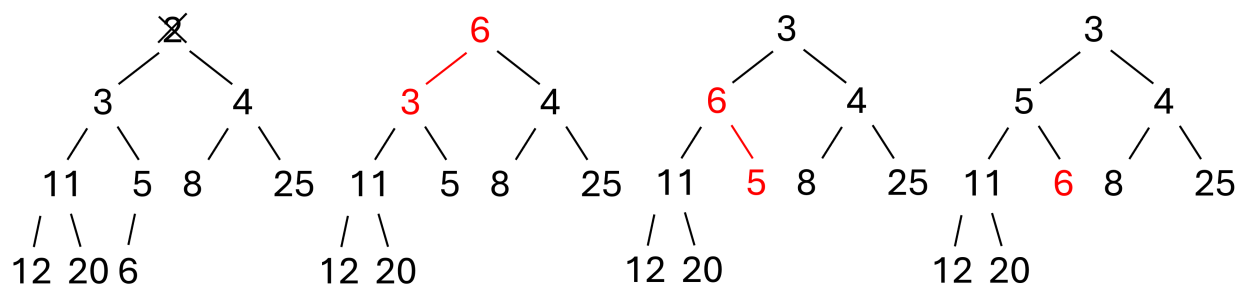
После такого просеивания свойство кучи восстановится. При просеивании вверх значение в сыне меняется со значением в отце, если это необходимо. Для всех поддеревьев, которые не лежат на пути обмена, свойство кучи сохраняется.

Для узлов, в которых происходили обмены свойство кучи также сохраняется. Действительно, значение в узле могло только уменьшиться, а значит для того, сына, которые не лежал на пути обменов, свойство кучи тоже сохранится.

Количество обменов при просеивании вверх не превосходит глубины (количества уровней) кучи, т.е. сложность этой операции составляет $O(\log N)$.

Теперь научимся выполнять операцию удаления минимума. Минимум находится в корне, после его удаления на его место должен попасть меньший из сыновей. Место сына освободится и его тоже нужно будет заполнять и так далее и так далее. Это приведет к тому, что куча перестанет быть плотной (когда все элементы, кроме, возможно, одного, имеют двух детей или не имеют детей вообще). Поэтому для удаления используется другой метод.

При удалении мы копируем в корень значение самого последнего элемента и просеиваем его вниз (sift-down). Последний элемент удаляется, детей у него гарантированно нет. При просеивании элемента вниз мы выбираем меньшего из детей и, если меньший из детей меньше текущего элемента, меняем их местами. Пока текущий элемент больше меньшего из своих детей, меняем их местами.

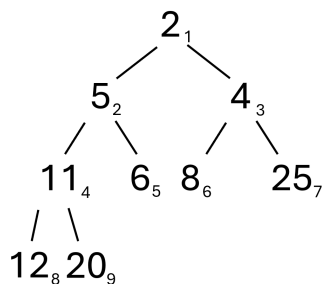


Просеивания элемента вниз также занимает $O(\log N)$, хотя и реализуется немного сложнее, чем просеивание вверх.

Реализация кучи есть в большинстве языков программирования. В python куча минимумов реализована в модуле `heapq`, однако стоит хотя бы раз написать реализацию кучи вручную.

4.3 Представление кучи в виде массива

Чаще всего узлы дерева хранятся как структуры, в которых есть поле для значения в узле, а также ссылки на детей. Однако благодаря тому, что в куча заполнена плотно, можно использовать более простое и эффективное решение — хранение дерева в массиве. Занумеруем все узлы кучи сверху-вниз слева-направо, начиная с единицы:



Этой куче соответствует массив:

0	1	2	3	4	5	6	7	8	9
-	2	5	4	11	6	8	25	12	20

Легко заметить такое свойство: номера детей узла i будут равны $2i$ и $2i + 1$. Номер предка элемента i равен $i//2$. У корня (элемента с индексом 1), предком будет элемент с индексом 0, который в кучу не входит и это может быть удобным признаком окончания процесса просеивания.

В принципе, можно использовать и 0-индексацию, тогда номера детей узла будут равны $2i + 1$ и $2i + 2$, а номер предка будет определяться как $(i - 1)//2$. У каждой из двух реализаций есть свои достоинства и недостатки.

Зная где находится корень дерева, а также правила определения предков и потомков, очень легко написать операции просеивания вверх и вниз. При этом не нужно пользоваться рекурсией, достаточно обычного цикла `while`.

4.4 Пирамидальная сортировка

С помощью кучи можно отсортировать массив: достаточно положить все элементы массива в кучу, а затем доставить из нее минимумы или максимумы (в этом случае

заполнять массив нужно будет справа-налево). Оба этих процесса займут $O(N \log N)$ и $O(N)$ дополнительной памяти. Однако существует способ сортировки с помощью кучи с меньшей константой и потреблением дополнительной памяти.

Пирамидальная сортировка (heapsort) — это алгоритм сортировки, основная идея которого состоит в том, чтобы превратить исходный массив в кучу максимумов, а затем последовательно извлекать максимальные элементы, помещая их в конец массива.

Процесс превращение массива в кучу максимумов называется "heapify" и устоявшегося русского термина для него нет.

В процессе выполнения heapify мы будем постепенно превращать массив в кучу максимумов с помощью просеивания вниз. Чтобы корректно просеивать элемент вниз, необходимо, чтобы оба сыновних поддерева уже являлись кучами максимумов. Если посмотреть на изначальный массив как на кучу, то это свойство выполнено только для листьев, которые находятся на последних $\lceil N/2 \rceil$ позициях в массиве. Эти элементы являются листьями, так как номера их детей больше размера кучи и, следовательно, дети у них отсутствуют.

Процесс просеивания вниз начнем с узлов, являющихся родителями листьев. Перебирать их можно, двигаясь по левой половине массива справа-налево (от $\lfloor N/2 \rfloor$ до 1 включительно, точные границы зависят от выбранного способа нумерации).

Просеивание выполняется до тех пор, пока свойство кучи не восстановится или мы не достигнем листа. То есть, точно так же как при просеивании вниз корня после удаления элемента, но начинается он не в корне, а в очередной перебираемой позиции.

Рассмотрим пример

$$A = [7, 2', 5, 2'', 9, 1].$$

Таблица ниже показывает, как последовательно вызывается просеивание вниз для узлов с индексами $i = 3, 2, 1$ (в порядке обхода справа-налево). В столбце «Обмен» указаны индексы, которые меняются местами.

Этап	Результат	Обмен
Исходный	$[7, 2', 5, 2'', 9, 1]$	—
1. $i = 3$	$[7, 2', 5, 2'', 9, 1]$	5 и 1 стоят правильно (нет обмена)
2. $i = 2$	$[7, 9, 5, 2'', 2', 1]$	$9 \leftrightarrow 2'$
3. $i = 1$	$[9, 7, 5, 2'', 2', 1]$	$9 \leftrightarrow 7$

После последнего шага массив является кучей максимумов: каждый узел не меньше своих детей. Обратите внимание на числа $2'$ и $2''$ — они одинаковые, но помечены метками, которые помогут нам определить устойчивость пирамидальной сортировки.

На первый взгляд может показаться, что сложность heapify составляет $O(N \log N)$ ($N/2$ просеиваний, каждое можно оценить сверху $O(\log N)$), но на самом деле она равна $O(N)$. Докажем это. Пусть H — высота кучи ($H = \lfloor \log_2 N \rfloor$). На уровне K (считаем от корня, $K = 0$) находится не более 2^K узлов, каждый из которых может спуститься не более чем на $H - K$ уровней. Общее число обменов при построении кучи оценивается суммой

$$\sum_{K=0}^{H-1} 2^K (H - K) = \sum_{D=1}^H D 2^{H-D} = O(N).$$

В формуле $D = H - K$ — количество уровней, на которое может опуститься узел уровня K . Сумма растёт линейно от N , а не от $N \log N$, поэтому построение кучи имеет сложность $\Theta(N)$. Даже если просеиваний не будет, мы все равно потратим порядка $N/2$ операций на проверки элементов из первой половины, именно поэтому мы можем говорить об оценке Θ .

Теперь у нас есть куча максимумов, и мы будем постепенно забирать из нее максимальные элементы и размещать их в конце массива. То есть в начале массива будет находиться куча, а в конце массива будет постепенно формироваться отсортированная часть массива.

После каждой операции удаления максимума будем, как обычно, копировать последний элемент кучи на в корень и просеивать его вниз. При этом необходимо помнить что куча это не весь массив, а только его префикс длиной $N - 1, N - 2, \dots, 1$.

Алгоритм выглядит так:

1. Построить кучу максимумов из $A[1 \dots N]$ (heapify).
2. Для $k = N \rightarrow 2$:
 - (a) Поменять местами $A[1] \leftrightarrow A[k]$ (наибольший элемент уходит в конец, последний элемент кучи помещается в корень).
 - (b) Уменьшить размер рассматриваемой кучи до $k - 1$.
 - (c) Выполнить просеивание вниз в массиве $A[1 \dots k - 1]$.

Рассмотрим пошагово как выполняется сортировка полученной после heapify кучи

$$A = [9, 7, 5, 2'', 2', 1].$$

Шаг	Обмен $A[1] \leftrightarrow A[k]$	Массив	Обмен в sift-down
0 (куча)	—	$[9, 7, 5, 2'', 2', 1]$	—
1, $k = 6$	$9 \leftrightarrow 1$	$[1, 7, 5, 2'', 2', 9]$	-
1, $k = 6$	-	$[7, 1, 5, 2'', 2', 9]$	$1 \leftrightarrow 7$
1, $k = 6$	-	$[7, 2'', 5, 1, 2', 9]$	$1 \leftrightarrow 2''$
2, $k = 5$	$7 \leftrightarrow 2'$	$[2', 2'', 5, 1, 7, 9]$	-
2, $k = 5$	-	$[5, 2'', 2', 1, 7, 9]$	$2' \leftrightarrow 5$
3, $k = 4$	$5 \leftrightarrow 1$	$[1, 2'', 2', 5, 7, 9]$	-
3, $k = 4$	-	$[2', 2'', 1, 5, 7, 9]$	$1 \leftrightarrow 2'$
4, $k = 3$	$2' \leftrightarrow 1$	$[1, 2'', 2', 5, 7, 9]$	-
4, $k = 3$	-	$[2'', 1, 2', 5, 7, 9]$	$1 \leftrightarrow 2''$
5, $k = 2$	$2'' \leftrightarrow 1$	$[1, 2'', 2', 5, 7, 9]$	-

На третьем шаге при просеивании 1 вниз оба ребенка были равны 2, мы выбрали правого ребенка $2'$, что привело к тому, что сортировка стала неустойчивой. Без запоминания исходного порядка (и, как следствие, использования дополнительной памяти) сделать пирамидальную сортировку устойчивой нельзя.

Оценим сложность пирамидальной сортировки. На каждой из $N - 1$ итераций вызывается просеивание вниз, которое в худшем случае спускает элемент на высоту текущей кучи $\log K$. Для первых $N/2$ элементов высота кучи неизменна, поэтому оценка в худшем случае только для этих элементов даст нам количество операций $N/2 \cdot \log N/2$.

Вместе с оценкой сверху (для каждого из N элементов $\log N$ просеиваний), получим оценку $\Theta(N \log N)$ в худшем случае.

Таким образом, пирамидальная сортировка работает за время $O(N \log N)$, потребляет $O(1)$ дополнительной памяти и очень хорошо подходит для случаев, когда выделение дополнительной памяти проблематично. Например, если сортируемый массив очень длинный или сортировка происходит на устройстве с небольшим объемом памяти (например, микроконтроллер для интернета вещей). На реальном компьютере константа у пирамидальной сортировки довольно большая, так как в куче не последовательный доступ памяти (индекс элемента массива увеличивается, примерно, в два раза), что приводит к промахам мимо кэша процессора (cache miss) и довольно медленной загрузке в кэш страницы из оперативной памяти.

4.5 Бинарный поиск

Идея бинарного (двоичного) поиска основана на том, чтобы на основе вопросов с ответом «да/нет» разделять множество примерно пополам, на каждом шаге сокращая область поиска вдвое.

Например, если один игрок загадал число X от 1 до 100, а второй игрок может задавать любые вопросы с ответом «да» или «нет», то можно разработать следующую стратегию генерации вопросов, которые при любом ответе будут сокращать область поиска примерно вдвое:

- Установим границы поиска $L = 1$, $R = 100$, границы включительно
- На каждом шаге будем выбирать середину отрезка поиска $M = (L + R)/2$. На первом шаге $M = 50$
- Зададим вопрос: верно ли, что названное число M больше либо равно чем загаданное число X ?
- Если ответ «да», то надо изменить правую границу поиска ($R = M$), потому что если мы назовем любое число, большее M мы гарантированно получим ответ «да» и задавать такие вопросы бесполезно
- Если ответ «нет», то надо изменить правую границу поиска, причем исключив само значение M ($L = M + 1$) / Значение M можно исключить, т.к. для него уже известен ответ «нет», а нам нужно найти самое маленькое такое M , что ответ для него «да»

В конце границы поиска станут равны ($L = R$) и бинарный поиск сойдется к первому числу, больше либо равному загаданному X .

Промоделируем процесс, если загадано число $X = 42$:

	М	Проверка	Границы
1	$M = (1 + 100) // 2 = 50$	$50 \geq 42? \rightarrow TRUE$	$L = 1, \mathbf{R} = \mathbf{M} = \mathbf{50}$
2	$M = (1 + 50) // 2 = 25$	$25 \geq 42? \rightarrow FALSE$	$\mathbf{L} = \mathbf{M} + \mathbf{1} = \mathbf{25} + \mathbf{1} = \mathbf{26}, R = 50$
3	$M = (26 + 50) // 2 = 38$	$38 \geq 42? \rightarrow FALSE$	$\mathbf{L} = \mathbf{M} + \mathbf{1} = \mathbf{38} + \mathbf{1} = \mathbf{39}, R = 50$
4	$M = (39 + 50) // 2 = 44$	$44 \geq 42? \rightarrow TRUE$	$L = 39, \mathbf{R} = \mathbf{M} = \mathbf{44}$
5	$M = (39 + 44) // 2 = 41$	$41 \geq 42? \rightarrow FALSE$	$\mathbf{L} = \mathbf{M} + \mathbf{1} = \mathbf{41} + \mathbf{1} = \mathbf{42}, R = 44$
6	$M = (42 + 44) // 2 = 43$	$43 \geq 42? \rightarrow TRUE$	$L = 42, \mathbf{R} = \mathbf{M} = \mathbf{43}$
7	$M = (42 + 43) // 2 = 42$	$42 \geq 42? \rightarrow TRUE$	$L = 42, \mathbf{R} = \mathbf{M} = \mathbf{42}$

L и R сошлись за 7 вопросов, для некоторых загаданных чисел будет достаточно 6 вопросов, но для поиска любого числа гарантированно хватит 7.

Код для реализации подобного бинарного поиска содержит функцию `check`, которая моделирует ответы на вопросы:

```
def check(m):
    return m >= x
```

```
l = 1
r = 100
while l < r:
    m = (l + r) // 2
    if check(m):
        r = m
    else:
        l = m + 1
```

Существует множество других реализаций бинарного поиска, например такие, в которых правая граница области поиска не включена. У них есть свои достоинства и недостатки. Можно выбрать любую реализацию бинарного поиска, но стоит убедиться, что бинарный поиск заканчивает свою работу, а не уходит в вечный цикл. Для этого нужно проверить идею на простом тесте: пусть у нас осталось только два элемента (в нашем случае это означает, что $R = L + 1$). Выберем M и проверим оба случая (если ответ функции `check` да или нет). При любом ответе бинарный поиск должен завершить свою работу с корректными значениями L и R .

Рассмотрим неверную реализацию бинарного поиска и проведем этот эксперимент.

```
def check(m):
    return m >= x
```

```
l = 1
r = 100
while l < r:
    m = (l + r) // 2
    if check(m):
        r = m
    else:
        l = m
```

Пусть $R = L + 1$, тогда $M = (L + R) // 2 = L$. В случае ответа «да» R станет равно M (и, следовательно L) и цикл корректно закончится. В случае ответа «нет»

$L = M = L$, то есть L останется без изменений и цикл будет работать вечно. Такой тест на двух соседних элементах позволяет найти большинство ошибок в реализации бинарного поиска.

Таким образом, можно сформулировать требования к задаче и правила ее решения с помощью бинарного поиска:

- Мы получаем ответ «да» для всех чисел больше либо равных X и ответ «нет» для всех меньших чисел
- Наша цель — найти число X (минимальное число, для которого мы получаем ответ «да»)
- Искомое число должно находиться между начальными значениями L и R , т.е. на отрезке поиска есть хотя бы один ответ «да». Если его нет, то границы бинарного поиска сойдутся к R

Сложность бинарного поиска составляет $\Theta(\log N)$, где N — длина отрезка поиска ($N = R - L + 1$). Определение сложности очевидно, т.к. каждый раз область поиска сокращается вдвое.

4.6 Бинарный поиск по ответу

Во многих задачах нам нужно найти минимальное значение параметра, удовлетворяющего какому-либо свойству. Если задача удовлетворяет свойству, что до числа X ответ «нет», а начиная с числа X ответ «да», то для ее решения можно применить бинарный поиск.

При этом может оказаться так, что у задачи есть другое решение, но проверка того, является ли ответ подходящим, намного проще этого решения. А значит, можно воспользоваться бинарным поиском и написать код гораздо быстрее.

Первый пример: В попечительском совете школы N человек из них K родителей. Требуется, чтобы не менее трети членов попечительского совета было родителями. Сколько родителей нужно добавить, чтобы их стало не менее трети?

Эту задачу можно решить с помощью формулы, однако решение с бинарным поиском намного проще. Действительно, допустим мы добавили M родителей, тогда в функции `check` достаточно проверить, что $K + M$ составляет не менее трети от $N + M$ (здесь часто забывают, что новые родители также прибавляются к общему числу членов попечительского совета и ошибочно используют N). Чтобы избежать деления и возможных проблем с округлением лучше воспользоваться умножением, тогда функция `check` будет возвращать результат вычисления логического выражения $3 \times (K + M) \geq (N + M)$. Если функция вернет `TRUE`, то добавленного количества родителей достаточно (будем менять R), в противном случае недостаточно (будем менять L).

Также нужно подобрать правильные начальные значения для левой и правой границ. Левая граница $L = 0$, ведь может оказаться так, что родителей уже достаточно и никого добавлять не нужно. Правую границу можно выбрать так, чтобы ответ был заведомо верным. Например, подходит значение $R = N$ — если добавить в попечительский совет столько родителей, сколько членов попечительского совета было изначально, то родителей в нем станет не меньше половины. Если подбирать значение R слишком

тщательно можно решить задачу с помощью формулы, что не является нашей целью при изучении бинарного поиска. Поэтому достаточно взять заведомо подходящее значение. Ничего страшного, если оно будет заметно больше правильного ответа: $\log$ растет так медленно, что это не окажет серьезного влияния на скорость работы программы.

Второй пример: Васе нужно решить N задач. В первый день он решает одну задачу, во второй две, в третий день три задачи и так далее: каждый день он решает на одну задачу больше, чем в предыдущий день. Сколько дней потребуется Васе, чтобы решить не менее N задач в сумме?

Эту задачу также можно решить аналитически. Фактически, нам необходимо определить количество членов арифметической прогрессии по известной сумме этой прогрессии. Однако гораздо проще решить обратную задачу: подсчитать сумму арифметической прогрессии по известному количеству ее членов. При этом мы сможем воспользоваться бинарным поиском: пусть Вася решал задачи M дней, тогда считаем сумму всех чисел от 1 до M и сравниваем ее с N . Эту сумму можно легко подсчитать с помощью известной формулы $\frac{M \times (M+1)}{2}$.

Подбираем значения для левой и правой границы. В качестве левой границы возьмем $L = 0$ (вдруг Васе не нужно решать задачи), а в качестве правой границы мы можем взять, например, число N . Действительно, даже если бы Вася каждый день решал всего лишь по одной задаче, то он бы решил необходимое количество задач. При этом нужно быть внимательными: при большом значении R в процессе вычисления суммы арифметической прогрессии мы будем получать числа порядка R^2 . На `python` это не критично, но на других языках может вызвать переполнение целочисленной переменной. Поэтому за типами переменных и значением R нужно следить внимательно.

Третий пример: у Васи есть квадратная доска размером $N \times N$. Ему необходимо разместить K квадратных стикеров с целочисленной длиной стороны на этой доске так, чтобы их размер был как можно больше. То есть, нужно найти такое максимальное M , что на доске размером $N \times N$ помещается не менее K стикеров размером $M \times M$.

Если бы нам был дан размер стикера M , то мы бы легко определили значение $K = \lfloor N/M \rfloor \times \lfloor N/M \rfloor$ — это количество рядов стикеров, умноженное на количество стикеров в ряду. Получается, что мы легко можем решать задачу «обратную» данной и попробуем приспособить бинарный поиск для решения исходной задачи.

В этой задаче функция имеет другой вид: пока стикеры достаточно маленькие они помещаются в достаточном количестве. А начиная с какого-то размера стикера они перестают помещаться. Таким образом, нам нужно найти последнее «хорошее» значение M вместо поиска первого «хорошего», как это было в предыдущих задачах.

Существует два подхода к решению этой задачи. Можно найти первое «плохое» значение, вычесть из него единицу и получить последнее «хорошее». А можно изменить логику бинарного поиска, чтобы он искал именно последнее «хорошее» значение.

Пусть в качестве M мы выбрали середину отрезка между L и R . Если `check(M)`, то нужно заменить левую границу ($L = M$, менять на $M + 1$ нельзя — вдруг мы попали в последний хороший элемент). В противном случае нужно заменить правую границу, причем можно сделать $R = M - 1$, потому что значение M «плохое».

Проведем тест на двух элементах. Если выбирать $M = (L + R) // 2$ и $R = L + 1$, то $M = L$. В случае, если `check(M)` вернет `TRUE`, то $L = M$ — вечный цикл, плохо. Если вернет `FALSE`, то $R = M - 1 = L - 1$ — тоже плохо, правый указатель стал меньше левого.

Эта проблема легко исправляется, если выбирать $M = (L + R + 1) // 2$ (это округление

вверх при делении на 2). Если осталось всего два элемента, то $M = R$. Тогда в случае *TRUE*: $L = M = R$, а в случае *FALSE*: $R = M - 1 = L$ — бинпоиск корректно сходится.

В этой задаче в качестве начальной левой границы можно взять $L = 1$, а в качестве правой $R = N$.

Четвертый пример: Вдоль прямой расположены N стойл, заданы их координаты $X[i]$ в порядке возрастания. Необходимо расставить в них K коров так, чтобы минимальное расстояние было как можно больше.

Будем решать эту задачу бинарным поиском по ответу: искать такое максимальное M , что можно расставить коров так, чтобы минимальное расстояние между ними было не меньше M . В этой задаче будет довольно сложное моделирование процесса расстановки. Самое левое стойло всегда выгодно занять, поставим туда корову. После этого пройдем по всем стойлам и будем ставить очередную корову в том случае, если расстояние от предыдущего занятого стойла M или больше. Если очередное стойло находится слишком близко к предыдущему занятому, то пропустим его. Таким образом, функция `check` в этой задаче будет работать за $O(N)$. А общая сложность алгоритма составит $O(N \log X)$, где X — расстояние между крайними стойлами.

Здесь нужно воспользоваться бинарным поиском таким же, как в предыдущем примере: искать последнее хорошее значение.

4.7 Бинарный поиск в массиве

Очень частой задачей, решаемой бинарным поиском, является поиск элемента в отсортированном массиве. Сформулируем задачу так: дан отсортированный по неубыванию массив A необходимо проверить, есть ли в нем элемент X .

Мы можем воспользоваться обычным бинарным поиском, придумав правильную функцию `check`. Будем искать такое минимальное M , что $A[M] \geq X$, т.е. функция `check` должна возвращать *TRUE* если $A[M] \geq X$.

После этого необходимо проверить, что значение $A[M] = X$. Если X в массиве нет, то бинарный поиск сойдется к первому большему чем X число. Таким образом, мы решили еще одну задачу: найти самое маленькое число, больше либо равное X . В C++ такая функция называется *lower_bound*, а в Python *bisect_left*.

Есть один неприятный момент: если X будет больше максимального числа в массиве, то бинарный поиск со включительными границами сойдется к последнему элементу (с индексом $N - 1$), который будет меньше X . Так произойдет, потому что ни для одного параметра от L до R значение функции `check` не будет хорошим. Этот случай необходимо обработать отдельно.

С помощью бинарного поиска можно решать и многие другие задачи по поиску чисел в массиве. Например, найти первое строго большее число или последнее меньше либо равное и так далее. Рассмотрим такую задачу: нам необходимо определить, сколько раз число X встречается в массиве. Если найти первое вхождение числа X и двигаться от него вправо до тех пор, пока число не станет отличаться от X или не дойдем до конца массива, то мы потратим $O(N)$ действий в худшем случае (например, когда весь массив состоит из чисел X).

Правильным решением этой задачи является два бинарных поиска: первый ищет индекс первого вхождения числа больше либо равного X , а второй — индекс последнего вхождения числа меньше либо равного X . Или второй бинпоиск может искать первое

вхождение числа строго большего X . В любом случае надо помнить о краевом эффекте, который проявится, если X равно максимальному числу в массиве и правильно учитывать этот случай.

4.8 Вещественный бинарный поиск

Бинарный поиск можно применять и для вещественных чисел. Например, если у нас есть строго монотонная функция, то мы можем найти такое X , что $F(X)$ будет равно заданному числу Y . При этом значение будет неточным, но в пределах некоторой погрешности. Для нестрого монотонной функции бинарный поиск может работать некорректно.

Рассмотрим пример: найти кубический корень числа $Y > 1$. Мы можем воспользоваться бинарным поиском, взяв в качестве левой границы $L = 0$, а в качестве правой $R = Y$. В качестве M будем выбирать вещественное число, равное $(L + R)/2$, возводить его в куб и сравнивать с заданным Y . Если $M^3 > Y$, то надо менять правую границу поиска, в противном случае — левую. При этом процесс должен остановиться, когда мы достигнем заданной точности. Например, если нам требуется найти кубический корень с абсолютной погрешностью не более 10^{-5} , то цикл должен выполняться пока разница между R и L больше 10^{-5} . Затем в качестве ответа можно выбрать середину отрезка между L и R , хотя, вообще говоря, подходит любое число из отрезка.

Приведем код для поиска кубического корня:

```
def check(m):
    return m * m * m >= y

l = 1
r = y
while (r - l) > 10**-5:
    m = (l + r) / 2
    if check(m):
        r = m
    else:
        l = m
```

Часто вместо подсчета разницы между R и L используют фиксированное количество итераций. Для этого необходимо взять максимально допустимую длину начального отрезка поиска, разделить на абсолютную погрешность и подсчитать от этого логарифм по основанию 2, округленный вверх. То есть, количество шагов бинарного поиска будет равно:

$$\lceil \log \frac{R - L}{\varepsilon} \rceil$$

Где ε — абсолютная погрешность, а L и R — начальные границы поиска в худшем случае.